

Truthiness

The ability to express true or false is vital in any programming language. It helps us build conditional logic and understand the state of an object or expression. Typically, we capture the notion of whether a value is true or false in a boolean data type. A boolean is a data type whose only purpose is to convey whether it is true or false.

Python uses the True and False boolean values. You can print them, assign them to variables, pass them around, and test them:

Copy Code

```
print(True) # True
print(False) # False

def make_longer(string, longer):
    if longer:
        return string + string
    else:
        return string

print(make_longer("abc", True)) # 'abcabc'
print(make_longer("xyz", False)) # 'xyz'

def is_digit(char):
    if '0' <= char <= '9':
        return True
    else:
        return False

print(is_digit("5")) # True
print(is_digit("a")) # False

value = True # or False

if value is True:
    print("It's true!")
elif value is False:
    print("It's false!")
else:
    print("It's not true or false!")
```

Expressions and Conditionals

In real code, you wouldn't usually use the True or False values directly in a conditional expression like `value == True`. Instead, you would merely evaluate an expression that should evaluate as either True or False. For instance:

Copy Code

```
num = 5

if num < 10:
    print("small number")
else:
    print("large number")
```

The above code outputs small number since `num < 10` evaluates as True. We can verify that in the Python REPL:

Copy Code

```
>>> num = 5
>>> num < 10
True
```

```
>>> num = 12
>>> num < 10
False
```

In much the same way, a function doesn't usually return True or False

explicitly. Instead, it returns the result of a conditional expression. For instance:

Copy Code

```
def is_small(number):  
    return number < 10
```

```
num = 15
```

```
if is_small(num):  
    print("small number")  
else:  
    print("large number")
```

This code prints large number since `is_small(num)` evaluates as `False` when `num` is greater than or equal to 10.

Logical Operators

Logical operators evaluate an expression that involves two subexpressions, then return a value that evaluates as `True` or `False`.

The and Operator

The `and` operator evaluates as `True` only when both sub-expressions evaluate as `True`:

Copy Code

```
print(True and True)    # True  
print(True and False)   # False  
print(False and True)   # False  
print(False and False)  # False
```

```
num = 5
```

```
print((num < 10) and (num > 3)) # True  
print((num < 10) and (num > 6)) # False  
print((num > 10) and (num < 6)) # False  
print((num > 10) and (num < 3)) # False  
Those results shouldn't be too surprising.
```

Note, though, that we didn't have to put sub-expressions like `num > 10` in parentheses. Python's precedence rules handle the comparison operators like `<` and `>` with higher precedence than logical operators like `and`. This means that `num < 10 and num > 3` is treated as `(num < 10) and (num > 3)`, not as `num < (10 and (num > 3))` or `num < ((10 and num) > 3)`.

The precedence rules can cause confusion, especially if your concept of how to evaluate an expression differs from Python's. Thus, it's a good idea to always use parentheses with expressions that involve multiple operators:

Copy Code

```
(num < 10) and (num > 3)
```

The parentheses make your intent clear and prevent confusing situations.

You can chain as many sub-expressions as you'd like with `and`; the sub-expressions get evaluated left to right. If any sub-expression is `False`, the entire `and` chain evaluates as `False`. The whole expression evaluates as `True` only when all of the sub-expressions evaluate as `True`:

Copy Code

```
num = 5
```

```
print((num < 10) and (num > 0) and ((num % 2) == 1)) # True  
print((num < 10) and (num > 0) and ((num % 2) == 1) and False) # False
```

The or Operator

`or` operator evaluates as `True` when either of the two sub-expressions evaluates as `True`; it evaluates as `False` when both sub-expressions evaluate as `False`.

Copy Code

```
print(True or True)      # True
print(True or False)     # True
print(False or True)     # True
print(False or False)    # False
```

```
num = 5
print((num < 10) or (num > 3)) # True
print((num < 10) or (num > 6)) # True
print((num > 10) or (num < 6)) # True
print((num > 10) or (num < 3)) # False
Again, those results shouldn't be too surprising.
```

The not Operator

The not operator is a bit different than the previous two operators. It simply inverts the truth value of the condition it's applied to. If a condition is True, not will make it False and vice versa.

Copy Code

```
print(not True) # False
print(not False) # True
```

```
value = 3
is_even = (value % 2 == 0)
```

```
print(is_even)      # False
print(not is_even)  # True
```

In essence, not serves as a logical negation. It's useful when you want to check the opposite of some condition.

Short-Circuit Operators

Both and and or exhibit a behavior called short-circuiting. That means that Python stops evaluating sub-expressions once it can determine the final value. In the case of and, Python short-circuits when it realizes that the entire expression can't be true; that is, when it encounters a false sub-expression. With or, it short-circuits when it realizes that the expression can't be false; that is, at least one sub-expression is true.

Consider and: it short-circuits when it encounters the first sub-expression (from left-to-right) that evaluates as False:

Copy Code

```
print(False and len(None)) # False
```

Notice that the above code doesn't generate a TypeError even though the code includes the sub-expression len(None). By itself, len(None) would raise a TypeError. However, since the left side of the and guarantees that the entire expression can't be true, the right side never executes. Instead, the expression evaluates as False.

Conversely, this code does raise an exception:

Copy Code

```
print(True and len(None))
```

```
# TypeError: object of type 'NoneType' has no len()
```

Likewise, the or operator short-circuits when it encounters the first True sub-expression, again based on left-to-right evaluation:

Copy Code

```
print(True or len(None)) # True
```

The above code doesn't raise a TypeError since or didn't evaluate the second sub-expression; it short-circuited when True evaluated as True.

Relying on the short-circuit behavior can be dangerous, but it's sometimes

handy. For instance, suppose you have a name variable that usually contains a string, but may sometimes be None. Before you can use the string's `isupper()` method, you must confirm that it isn't None:

Copy Code

```
if name != None and name.isupper():
    print(f"Hi, {name}.")
else:
    print("Hello, whoever you are.")
```

This type of conditional expression often appears in real-life Python code, so get familiar with it. In particular, make sure that you understand that Python doesn't evaluate the right side of a short-circuit operator when the expression short-circuits.

Truthiness

The discussion so far should be mostly review. It's time to talk about the main topic of this assignment: what is truthiness, and how does it impact our code? We'll tackle that next.

After that review of booleans and logical operators, we're finally able to talk about the notion of truthiness. Truthiness differs from boolean values in that Python evaluates almost all values as true. There are some exceptions, however:

```
False
None
0
0.0
0j
"" (an empty string)
[] (an empty list)
{} (an empty dictionary)
() (an empty tuple)
set() (an empty set)
frozenset() (an empty frozenset)
range(0) (an empty range)
All of these values evaluate as false. Memorize them!
```

Notice that we've repeatedly used the phrases evaluated as true and evaluated as false. You can also use the terms `truthy` and `falsy` to describe the nature of the values. Be careful to make the same distinction in your own written and spoken communications. Saying that an expression returns `True` or `False` is not the same as saying that it returns a `truthy` or `falsy` value, or that it evaluates as `True` or `False`. The terms `True` and `False` refer to the Boolean values `True` and `False`; the other phrases refer to truthiness, that is, a `truthy` or `falsy` value.

Truthiness means that we can use any condition or logical expression:

Copy Code

```
num = 5
if num:
    print("valid number")
else:
    print("error!")
```

If you didn't know Python at all, you might guess that the above code should either output `error!` or generate an error of its own. However, when you run that code, it outputs `valid number`. That's because Python considers any non-zero number to be `truthy`. It does not, however, mean that `num` is equal to `True`:

Copy Code

```
print(num == True) # => False
```

The use of `truthy` and `falsy` values sometimes leads to some surprising code:

Copy Code

```
name = get_name_from_user()
```

```
if name:
    print(f"Hi {name}")
else:
    print("you must enter your name!")
```

Presumably, `get_name_from_user` solicits and returns the user's name and returns an empty string `""` if the user doesn't enter a name. Since `""` is falsy, we can test for a missing name by evaluating `name`. If the user didn't enter her name, the expression evaluates as `False`, and the code prints `you must enter your name!`.

While this type of code is commonplace, most style guides strongly discourage it. We also discourage it. It's safer and more intentional to write:

Copy Code

```
name = get_name_from_user()
if name == "":
    print("you must enter your name!")
else:
    print(f"Hi {name}")
```

Python Virtual Environments

Throughout Launch School's Python courses, we strongly recommend using virtual environments to manage your project's dependencies and to isolate your Python environments. Virtual environments provide a way to create separate, self-contained environments for different projects, ensuring that dependencies do not conflict and your projects remain organized.

By using virtual environments, you can:

- Manage project-specific dependencies without affecting the global Python environment.

- Avoid conflicts between different projects that require different library versions.

- Keep your development environment clean and portable.

- Ensure consistent development environments across different machines.

- Use the newest versions of Python (3.12 and up).

Version Notes

This assignment asks you to install specific versions of Python (3.9 and 3.11) and Flask (versions 2.3.1 and 2.1.2). If you receive a deprecation warning when installing one of these, you may ignore it. Deprecation doesn't mean you can't use it, especially for learning purposes.

If you're using Homebrew to install one of the older Python versions, you may get a message that looks like this:

Copy Code

```
Warning: No available formula with the name "python@3.9"
```

If this happens, please let us know so we can update the course. However, please don't wait for us to catch up; just use another version of Python. The specific Python versions you use are not important for this assignment. Our main goal is to show you how to setup and use virtual environments.

Overview of Virtual Environments

By now, you're used to running this command in the terminal to execute Python programs:

Copy Code

```
python my_program.py
```

This command executes the python program and feeds it the program in the `my_program.py` file. You can also type `python --version` to see what version of Python you're running. It may be 3.7.16, 3.11.2, or some other version. The types of programs you've been writing thus far should run correctly on any version greater than or equal to 3.7.1.

Suppose you have three different Python programs, one that requires version 3.9 of Python and two that require version 3.11. Suppose further that the two programs that need version 3.11 also require the Flask package, but one requires Flask version 2.3.1 while the other requires version 2.1.2. (Don't worry about what Flask does; you'll learn about Flask in the Core Curriculum.)

Program a.py requires version 3.9 of Python:

a.pyCopy Code

```
import sys

print(f"This is program {__file__}")
print(f"The Python version is {sys.version}")
Program b.py requires version 3.11 of Python and version 2.3.1 of Flask:
```

b.pyCopy Code

```
import sys
import flask

print(f"This is program {__file__}")
print(f"The Python version is {sys.version}")
print(f"The Flask version is {flask.__version__}")
Program c.py requires version 3.11 of Python and version 2.1.2 of Flask:
```

c.pyCopy Code

```
import sys
import flask

print(f"This is program {__file__}")
print(f"The Python version is {sys.version}")
print(f"The Flask version is {flask.__version__}")
Remember, the specific versions of Python are not important. The main requirement is that the a.py program should rely on an earlier Python version than the b.py and c.py programs do.
```

Likewise, you may need to use some other version of Flask than the ones we mention below. Just use two different versions in programs b.py and c.py.

Working with these programs would be tricky and tedious. Every time you use one of these programs, you must configure your system with different versions of Python and Flask. It would be much too complicated and time-wasting to uninstall and reinstall the different versions of Python and Flask.

A better solution would be to put the different versions of Python and Flask in different areas of your system. If you want to run a.py, you can modify the PATH variable to point to where the various Python 3.9 programs live. If you later want to run b.py, you can modify PATH to point to a location that contains both Python 3.11 and Flask 2.3.1. Finally, to run c.py, modify PATH so the system can find Python 3.11 and Flask 2.11.

That's a somewhat workable solution, but it's also a hassle to continually modify your PATH. A better solution would take care of the PATH variable semi-automatically. That's where Python's virtual environments come into play. Virtual environments let you set up multiple environments, each providing a specific version of Python and any required packages. When you run your program, you activate the virtual environment the program requires. That takes care of the necessary PATH changes.

Setting Up Virtual Environments

Creating a virtual environment for an application is a relatively straightforward process. First, decide on a centralized location for your virtual environments. We recommend using the conventional `~/venv` directory.

First, you should create the `~/venv` directory:

Copy Code

```
mkdir ~/.venv
```

You can disregard any errors that say that `.venv` already exists. That just means you've already created it.

Once you have this directory, you can create virtual environments for each program that requires a different configuration:

Install the version of Python you wish to use. See our Python book for details on installing Python.

Create a virtual environment with your desired Python version.

Activate the virtual environment you want to use. Activating an environment updates your `PATH` and other environment variables automatically.

Install any Python packages you want to use with the application.

For instance, let's create virtual environments for Python 3.9 and 3.11.

You do not need to memorize the commands for setting up virtual environments. You probably will memorize them if you use them often enough. Until then, however, you can always refer to this assignment.

Using Virtual Environments

This section applies to everybody.

Once your virtual environment is set up and your application is ready to run:

Activate the desired virtual environment (if it isn't already activated).

Run the application

Deactivate the virtual environment (or activate another virtual environment).

Using env_a

```
source ~/.venv/env_a/bin/activate # Step 1
```

```
python a.py # Step 2
```

```
deactivate # Step 3
```

Using env_b

```
source ~/.venv/env_b/bin/activate # Step 1
```

```
python b.py # Step 2
```

```
deactivate # Step 3
```

Using env_c

```
source ~/.venv/env_c/bin/activate # Step 1
```

```
python c.py # Step 2
```

```
deactivate # Step 3
```

You can also create shell scripts to run both commands more easily. However, that is beyond the scope of this lesson.

The Python Package Index

The Python Package Index, commonly abbreviated as PyPI, is a repository for many software tools and libraries you use with or in Python. You can also distribute your code via PyPI, and many Pythonistas do. For now, we will concern ourselves only with downloading software for your own use.

Python has provided a package manager called `pip` (Pip Installs Packages) since 2008. `Pip` provides the interface you need to install and manage packages from PyPI. While `pip` can install packages globally, it's best practice to use it within a virtual environment to avoid conflicts between project dependencies.

Use the `pip` command to install packages. For example, the following command will install the `example2` package:

Copy Code

```
pip install example2
```

Yes, you can run this command!

On some systems, you may not have a `pip` command. If you get a command not found error message, try `pip3` instead.

To uninstall a package, use pip with the uninstall subcommand. For instance, to uninstall the example2 package, you run:

Copy Code

```
$ pip uninstall example2
Found existing installation: example2 0.1
Uninstalling example2-0.1:
  Would remove:
    /Users/xyzzzy/.venv/env_b/lib/python3.10/site-packages/example2-0.1.dist-
info/*
    /Users/xyzzzy/.venv/env_b/lib/python3.10/site-packages/example2/*
    /Users/xyzzzy/.venv/env_b/lib/python3.10/site-packages/example2_module/*
Proceed (Y/n)?
```

The Would remove: output shows you what pip will uninstall. This safety measure helps prevent the removal of packages that may be used by other installed packages. If everything looks okay, you may continue the deletion by entering Y followed by Enter or Return.

If you want to see what packages are currently installed in your Python environment, you can use pip list:

Copy Code

```
$ pip list
Package      Version
-----
pip          23.0.1
setuptools   65.5.0
```

You can display some additional information about a specific package by using pip show:

Copy Code

```
$ pip show setuptools
Name: setuptools
Version: 68.1.2
Summary: Easily download, build, install, upgrade, and uninstall Python packages
Home-page: https://github.com/pypa/setuptools
Author: Python Packaging Authority
Author-email: distutils-sig@python.org
License:
Location: /opt/homebrew/lib/python3.11/site-packages
Requires:
Required-by:
pip used to have a built-in search capability for searching PyPI. However, PyPI decided to prevent searches by pip. If you try to use pip search now, you'll see a message that tells you to use the website search tool instead.
```

For now, understand that throughout this book and your journey as a Python developer, you may be required to pip install <some_package>. All it's doing is going out to PyPI and pulling down the appropriate packages for you to use on your local machine.

Installing Jupyter

As discussed in the Python book, JupyterLab and Jupyter Notebook are alternative Python REPLs. The most significant benefit provided by these programs is that you can write and try out code, copy and paste code with blank lines and indentation, and much more. The copy-and-paste functionality is especially handy as the built-in Python REPL badly mishandles blank lines in indented code when pasting. The built-in REPL is effectively unusable when you want to copy and paste such code.

You are not required to install or use JupyterLab or Jupyter Notebook. Aside from this assignment, we do not discuss or use either program in the Core Curriculum. They are merely tools that may prove useful to some students.

Note that we don't recommend installing Jupyter on GitHub Codespaces or AWS Cloud9; the process is currently too complicated. If you are using a virtual environment, you can use the Try Jupyter page to run "lite" versions of Jupyter in your browser.

If you've already installed Jupyter on your Mac by using Homebrew, we recommend uninstalling it now. It's better to install Jupyter in a virtual environment so you can use it with different versions of Python:

Copy Code

```
brew uninstall jupyterlab
```

The rest of this section provides instructions that should work on macOS, most versions of Linux, and Windows with WSL2.

To install JupyterLab in your virtual environments, first activate one of your virtual environments. Here, `VENVNAME` is a placeholder for whatever your virtual environment is named:

Copy Code

```
# VENVNAME is your virtual environment's name
source ~/.venv/VENVNAME/bin/activate
Now, use pip (or pip3) to install jupyterlab:
```

Copy Code

```
pip install jupyterlab
```

Repeat the last 3 commands for each of your virtual environments where you want to run JupyterLab. If you create new virtual environments later, you'll want to install JupyterLab in those environments as well.

You can now run JupyterLab with the following command:

Copy Code

```
jupyter lab
```

JupyterLab will open in your default browser. You can use the interface to create a new notebook and write some code. When you're ready to execute the code, just press `{Shift}{Enter}`. You can go back to the code, edit it, and run it again.

Pseudocode

When you write programming code, you're writing it for other programs to process. When you write Python, you are writing for the Python engine/interpreter to process. If you make a syntax error, the Python interpreter will complain, saying it doesn't know how to process the broken syntax. If there are no errors, the interpreter can parse and execute the code. Since programming code must be error-free, you must follow a rigid format when programming. A missing period or a wrong indentation may cause the entire program to break.

Pseudocode, on the other hand, is for humans -- machines or programs can't read it (at least, not yet!), so its format is relaxed. Human brains are far more flexible and powerful than programming language interpreters or compilers.

For example, here's some pseudocode for a function that determines which number in a collection has the greatest value.

Given a collection of numbers.

Iterate through the collection one by one.

save the first value as the starting value.

for each iteration, compare the saved value with the current value.

if the current number is greater

reassign the saved value as the current value

otherwise, if the current value smaller or equal
move to the next value in the collection
After iterating through the collection, return the saved value.
This approach is one that we can use to solve the problem. We don't start
writing code from the beginning, but instead try to load the problem into our
brain first.

When you first approach any problem, it's important to try to understand it
well. To do that, you must spend some time to load the problem into your brain.
Only then can you start to dissect it, understand it, and come up with an
execution path to solve it.

Unfortunately, loading the problem into your brain takes some effort, and you
must turn the problem over and over before it gets fully absorbed by your brain.
Doing this while working with a programming language is hard since your train of
thought gets interrupted time and again by having to deal with syntax issues.
Instead of thinking about the logic and dissecting the problem, you're searching
for specific language syntax issues to please the interpreter, which has nothing
to do with the logical aspect of solving a problem.

Therefore, there are two layers to solving any problem:

The logical problem domain layer.

The syntactical programming language layer.

When you're not yet fluent in a programming language, doing both at the same
time can be very difficult and frustrating.

Pseudocode comes into play at this point. Using pseudocode lets us focus on the
logical problem domain layer without dragging us down to the programming
language layer.

The problem with pseudocode, however, is that we cannot verify its logic. The
above pseudocode, for example, seems about right. However, how can we be sure?
To verify the logic, we must translate the pseudocode into programming code,
which is where you can focus on programming language syntax issues without
having it interrupt your flow.

Formal Pseudocode

Before we can take our pseudocode and translate it to program code, we must
formalize the pseudocode a little more. We'll still use English, but we'll use
some keywords to help us break down the program logic into concrete commands,
which makes translating to code more natural.

We'll use the below keywords to assist us, along with their meaning.

Keyword	Meaning
START	start of the program
SET	set a variable that we can use for later
GET	retrieve input from user
PRINT	display output to user
READ	retrieve a value from a variable
IF/ELSE IF/ELSE	show conditional branches in logic
WHILE	show looping logic
END	end of the program

We can use the above keywords to act as a pseudo-programming language, but one
that's still written in English, which lets us be more relaxed about the
precision of the syntax. Here's a stab at translating to formal pseudocode:

Copy Code

START

Given a collection of integers called "numbers"

SET iterator = 1

```

SET savedNumber = value within numbers collection at space 1

WHILE iterator <= length of numbers
    SET currentNumber = value within numbers collection at space "iterator"
    IF currentNumber > savedNumber
        savedNumber = currentNumber

    iterator = iterator + 1

```

PRINT savedNumber

Note that we're using PRINT to show the final return value. This translation almost looks like program code, but it's not. The advantage of this additional step is to give more structure to the pseudocode and to let us think at a more detailed level, yet still not worry about a programming language syntax. Though detailed it may be, this pseudocode still suffers from the same problem -- we can't verify that this logic is sound. Finally, to test the logic, we need to translate it into program code.

Translating Pseudocode to Program Code

We're using Python, so here's a stab at it in Python. Note that we are starting iterator at index 0 since list indexing in Python starts at 0. For the same reason, on line 5, we are using < instead of <=

Copy Code

```

def find_greatest(numbers):
    iterator = 0
    saved_number = numbers[iterator]

    while iterator < len(numbers):
        current_number = numbers[iterator]
        if current_number > saved_number:
            saved_number = current_number

        iterator += 1

    return saved_number

```

If we run the above code, we can show that our pseudocode logic works!

Now, let's look at the working code and start to improve it from a lower layer -- at the programming language level. For instance, what should we do if numbers is None? Perhaps we can use a guard clause that returns None, like this: if numbers is None: return. Now that we have the general logic and code in place, there are other small improvements we can make.

In this example, the function we wanted to write was straightforward. We were able to write a few lines of pseudocode, move it to a more formal pseudocode, and then translate it to Python. However, most problems you encounter will be more difficult than this example. You won't be able to take the same approach. That is, you won't be able to detail the entire problem in pseudocode, then translate it into Python. If you did, you'd likely discover that a lot of your pseudocode logic and assumptions are incorrect. You may need to make changes that ripple across the entire program, forcing you to start over time and again. Remember, pseudocode is a guess at the solution; there's no verification that the logic is correct. You can't do that until you translate it to program code.

For more sophisticated problems, we need to take a piecemeal approach when writing pseudocode, then translate that pseudocode to Python. Once we verify that the logic is correct, we can move to the next piece of the problem. Step-by-step, we slowly load the problem into our brain, verifying the logic each step along the way.

We'll show you how to use flowcharts to help with this in the next assignment. For now, try a few practice rounds using pseudocode to guide your problem-solving logic. For example, write out pseudocode (both casual and formal) that

does the following:

a function that returns the sum of two numbers

a function that takes a list of strings, and returns a string that is all those strings concatenated together

a function that takes a list of integers, and returns a new list with every other element from the original list, starting with the first element. For instance:

Copy Code

```
every_other([1,4,7,2,5]) # => [1,7,5]
```

a function that determines the index of the 3rd occurrence of a given character in a string. For instance, if the given character is 'x' and the string is 'axbxcdxex', the function should return 6 (the index of the 3rd 'x'). If the given character does not occur at least 3 times, return None.

a function that takes two lists of numbers and returns the result of merging the lists. The elements of the first list should become the elements at the even indexes of the returned list, while the elements of the second list should become the elements at the odd indexes. For instance:

Copy Code

```
merge([1, 2, 3], [4, 5, 6]) # => [1, 4, 2, 5, 3, 6]
```

You may assume that both list arguments have the same number of elements.

You don't need to write any Python code here; just practice writing the logic in English.

You don't need to use pseudocode for every bit of code you write, especially once you get down to the function level. However, it's a good idea to always use it here in this course and the associated Small Problem exercises. This will help you overcome problems in the short term, and prepare you for the interview assessment later on.

For brevity, we won't use pseudocode extensively in this course. However, we will use it for the more complicated problems and when we feel that it most helpful to see the pseudocode for a problem.

A Larger Problem

Now, let's suppose that the above flowchart is mapping out a solution to part of a more complex problem. Let's suppose that we need to ask the user to enter N collections of numbers and that we want to find and display the largest number from each collection. How would we approach coming up with a solution here?

Let's try to write some high-level pseudocode for this:

```
while the user wants to keep going
ask the user for a collection of numbers
extract the largest one from that collection and save it
ask the user if they want to input another collection
return the saved list of numbers
```

That seems reasonable, but you can see that the line extract the largest one from that collection is a sub-process that itself contains much logic. That functionality is self-contained, so it's a great candidate for a sub-process. We can, of course, turn that sub-process into a function. Let's not think about functions yet, though. Instead, let's keep our train of thought at the logical level and not think about code.

Now, you may be thinking that we were only able to see the "extract the largest one from that collection" as a sub-process since we did the previous assignment first. If we were tackling this larger problem first, it might not be apparent. That's a fair point. In that scenario, we probably must include the full pseudocode from the previous assignment, like this:

```
while the user wants to keep going
```

```
ask the user for a collection of numbers
iterate through the collection one by one.
```

```
save the first value as the starting value.
for each iteration, compare the saved value with the current value.
if the saved value is greater than or equal to the current number
move to the next value in the collection
otherwise, if the current value is greater than the saved value
reassign the saved value as the current value
after iterating through the collection, save the largest value into the list.
ask the user if they want to input another collection
return the saved list of numbers
```

When pseudocode gets long, it becomes difficult to trust the accuracy of the logic (remember, you can only verify the logic by running the actual program code). Therefore, it's prudent to extract a logical grouping into a sub-process and tackle the various pieces separately.

Let's take our shortened pseudocode from the beginning and translate this into formal pseudocode.

Copy Code

START

```
SET large_numbers = []
```

```
SET keep_going = True
```

```
WHILE keep_going == True
```

```
    GET "enter a collection"
```

```
    SET collection
```

```
    SET largest_number = SUBPROCESS "extract the largest one from that  
collection"
```

```
    large_numbers.append(largest_number)
```

```
    GET "enter another collection?"
```

```
    IF "yes"
```

```
        keep_going = True
```

```
    ELSE
```

```
        keep_going = False
```

```
PRINT large_numbers
```

Notice that we have a SUBPROCESS keyword to show that some other thing will extract the largest number from a collection. As before, we could have included the entire formal pseudocode from the previous assignment, but that would have made it very long. Our confidence in a large block of pseudocode can't be very high. Extracting logic into sub-processes lets us focus on a well-defined, narrowly-scoped set of pseudocode.

The interesting part of this flowchart is the processing square in the middle `num = find_largest(collection)`. It's our sub-process. You can think of this square as the zoomed-out high-level view of the very first flowchart from the top of this assignment.

Interestingly, when we move logic to sub-processes, we use a declarative type of syntax, rather than imperative. In other words, we can say "find_largest," rather than outline step by step how the logic should be. Thinking about how the high-level logic flows lets you create sub-processes to narrow the scope of your application. From a high level, writing declarative code fragments our program into logical sections, letting us focus on one section at a time. For example, if we wanted to add a validation feature to our program, we could have a sub-process that returns True or False given an input. We can call it `validate_input` and represent it as a square in our flowchart, without having to lay out the step-by-step imperative logic that's required to validate the user's input. From a high level, we can trust that this sub-process will do its job -- it only returns True or False. When we're ready to work on the logic in that `validate_input` sub-process, we can focus on the responsibilities of this sub-process and ignore the rest of the program.

As you use pseudocode and flowcharts to help you dissect the logic of a problem, you always need to think about how detailed the chart and words should be, and what you can extract to sub-processes. A programmer must always think about that when designing the solution to a problem. You won't get it right the first time.

Start at a high level, using declarative syntax. For example, if you're working on a calculator, you can start with something like this:

```
Get the first number
Make sure it's valid, otherwise, ask for another
Get the second number
Make sure it's valid, otherwise, ask for another
Get the operator
```

```
Make sure it's valid, otherwise, ask again
Perform the operation on the two numbers
```

```
Display result
```

```
Ask whether the user wants to do another calculation
```

In the above, you're not yet outlining exactly how to validate the inputs. No specifics or imperative style pseudocode yet. Once you have the high-level steps, it's time to drill down a level into imperative pseudocode and outline specifics.

This is going to take some practice, especially if you're new to programming. If you have trouble "thinking like a computer," try using these tools before you jump straight into program code.

In the long-term, you may not need pseudocode or flowcharts at all. However, on our interview coding assessments, we expect you to lay out your logic before you begin coding. Pseudocode is the most straightforward way to do that, so take time to practice pseudocode before you take those assessments.

The interesting part of this flowchart is the processing square in the middle `num = find_largest(collection)`. It's our sub-process. You can think of this square as the zoomed-out high-level view of the very first flowchart from the top of this assignment.

Interestingly, when we move logic to sub-processes, we use a declarative type of syntax, rather than imperative. In other words, we can say "find_largest," rather than outline step by step how the logic should be. Thinking about how the high-level logic flows lets you create sub-processes to narrow the scope of your application. From a high level, writing declarative code fragments our program into logical sections, letting us focus on one section at a time. For example, if we wanted to add a validation feature to our program, we could have a sub-process that returns True or False given an input. We can call it `validate_input` and represent it as a square in our flowchart, without having to lay out the step-by-step imperative logic that's required to validate the user's input. From a high level, we can trust that this sub-process will do its job -- it only returns True or False. When we're ready to work on the logic in that `validate_input` sub-process, we can focus on the responsibilities of this sub-process and ignore the rest of the program.

As you use pseudocode and flowcharts to help you dissect the logic of a problem, you always need to think about how detailed the chart and words should be, and what you can extract to sub-processes. A programmer must always think about that when designing the solution to a problem. You won't get it right the first time.

Start at a high level, using declarative syntax. For example, if you're working on a calculator, you can start with something like this:

```
Get the first number
```

Make sure it's valid, otherwise, ask for another
Get the second number
Make sure it's valid, otherwise, ask for another
Get the operator

Make sure it's valid, otherwise, ask again
Perform the operation on the two numbers

Display result

Ask whether the user wants to do another calculation

In the above, you're not yet outlining exactly how to validate the inputs. No specifics or imperative style pseudocode yet. Once you have the high-level steps, it's time to drill down a level into imperative pseudocode and outline specifics.

This is going to take some practice, especially if you're new to programming. If you have trouble "thinking like a computer," try using these tools before you jump straight into program code.

In the long-term, you may not need pseudocode or flowcharts at all. However, on our interview coding assessments, we expect you to lay out your logic before you begin coding. Pseudocode is the most straightforward way to do that, so take time to practice pseudocode before you take those assessments.

Debugging

Debugging is really at the core of what we do as programmers. While the glory and marketing go to shipping a finished application, the fact is that most of the day-to-day life of a programmer is spent stuck on some problem. Programmers can get through the trivial code rather quickly, so the majority of their time is spent analyzing and understanding a problem, experimenting or coming up with an approach, or debugging bugs in the code.

In this assignment, we'll focus on the general act of debugging, not working with a specific debugger tool. In a later assignment, we will briefly touch on the pdb debugger that ships with Python.

Temperament

When thinking about what attributes make a good programmer, one thing comes to mind: temperament. If the key to programming is debugging, then the key to debugging is a logical mind and patient temperament. Think Spock from Star Trek. Some people naturally possess this temperament, and that's an advantage when learning to program. Some people do not, so they must learn to develop this temperament. Natural intelligence helps, but, from our experience, most people have the intellectual capacity to become an application developer. What they need to do is develop a systematic, patient temperament when faced with a problem.

When debugging is required, something must be broken. When things break, it doesn't make you feel good. Programming is dealing with a constant stream of broken things and learning to deal with those ill feelings. Therefore, it's essential to gauge how you usually respond to problems. For example, if you're walking to the bus stop and the bus leaves right before you get there, what's your natural reaction? Someone who has the programmer temperament should figure out 1) when the next one comes, or 2) if there's an alternative path of transfers you can take, or 3) other alternative forms of transportation. If your reaction is a sinking feeling and frustration, you'll have to learn to deal with that, then transition to a systematic search for a solution. Fortunately, this can be learned, but remember to stay even-keeled and be systematic.

Dealing with feelings of frustration is a critical aspect of learning to program.

For example, click the button to see an example stack trace.

Stack Trace

The stack trace shows that the error occurred in the `filter_even_numbers` function on line 2 when attempting to iterate over the `numbers` collection. The error message `'NoneType' object is not iterable` indicates that the `numbers` collection is of type `NoneType`, which is not iterable.

A word of advice: use your stack traces. Make sure you understand what they are saying, and look at the code they identify as the failure point. If you don't use the trace, you may introduce more problems in the code or, worse yet, "fix" code that already works. The stack trace lets you focus on the right part of the program.

Online Resources

Search Engines After identifying the error message, take action. Study it, trace back through code, and understand how the program reached that error. Consider data involved and possible issues. Then search the error message.

For example, with `TypeError: 'NoneType' object is not iterable`, search for that exact phrase. Avoid system- and program-specific terms. To refine results, consider adding a language prefix, such as `"Python TypeError: 'NoneType' object is not iterable"`.

Stack Overflow Stack Overflow is a rich treasure trove of answers to common problems. Many of their answers rank highly in search engines, but sometimes it's worth searching on SO directly.

Documentation Finally, don't hesitate to consult the Python Documentation for core language insights, excluding framework specifics.

Steps for Debugging

The debugging process varies tremendously from person to person, but below are some steps you can follow until you've developed and honed habits of your own.

Reproduce the Error

The first step in debugging any problem is usually reproducing the problem. Programmers need a deterministic way to reproduce the problem consistently, and only then can they start to isolate the root cause. There's an old joke where programmers say that "it works on my machine" since they can't reproduce an error that occurs in the production environment. Reproducing the problem becomes more critical as you build more sophisticated applications with various external dependencies and moving parts. Reproducing the exact error is often more than half the battle in many tricky situations.

Determine the Boundaries of the Error

Once you can consistently reproduce the problem, it's time to tweak the code or data that caused the error. For example, in the previous assignment, we looked at a stack trace that was generated by this code:

Copy Code

```
return list(filter(lambda x: x % 2 == 0, numbers))
```

What might be wrong here? Are the arguments to `filter` given in the correct order? What does `lambda` do? Is `numbers` a sequence of numbers, or something else like a string or `None`? Does `filter` return an object that can be converted to a list? Perhaps we initialized the elements of `numbers` from an external source such as a file or server, but the external source returned no values?

Looking at the final line of the stack trace, we see that a `TypeError` was raised, with an explanation saying that `'NoneType' object is not iterable`. This suggests that `numbers` may have a value of `None`. Why is that the case? Perhaps we initialized `numbers` from something that returned `None` if there were no numbers available. If that's the case, we should specifically check whether `numbers` is `None`, and return an appropriate value (such as an empty list) if it is.

The filter function and the lambda keyword aren't important for our purposes at this time. We'll discuss them in a later course.

Trace the Code

Once you understand the boundaries of the problem, it's time to trace the code. Let's use a new example.

Copy Code

```
def process_student(student_data):
    name = student_data.get('name')
    grade = student_data.get('grade')
    return (name, grade)

def average_grade(grades):
    total = sum(grades)
    average = total / len(grades)
    return average

students = [
    {'name': 'Alice', 'grade': 85},
    {'name': 'Bob'}, {'name': 'Jack', 'grade': 72},
    {'name': 'Jane', 'grade': 75},
]

def collect_grades(students):
    grades = []
    for student in students:
        name, grade = process_student(student)
        grades.append(grade)
    return grades

grades = collect_grades(students)
print(average_grade(grades))
# TypeError: unsupported operand type(s) for +: 'int'
# and 'NoneType'
This code is relatively straightforward. It defines two functions
process_student which takes a student_data dictionary, an average_grade function
which calculates the average grade of all students and a collect_grades function
which groups all grades from students.
```

On line 25, we get an error when we try to calculate average grades.

The first step is to try to reproduce the problem consistently, and this code is already throwing an error every time we run it. That means we can go ahead and start trying to determine the boundaries of the error. Let's try to reduce the input size to just one student so we can track the bug more easily.

Copy Code

```
# Insert this next line below the current `students` assignment.
students = [{'name': 'Alice', 'grade': 85}]
```

```
# rest of the code omitted
```

```
print(average_grade(grades)) # 85
For the first student, everything works.
```

Copy Code

```
students = [{'name': 'Bob'}]
```

```
# rest of the code omitted
```

```
print(average_grade(grades)) # TypeError: unsupported operand type(s) for +:
'int' and 'NoneType'
```

However, for the second student, that doesn't have a grade, we get the same error. Let's try it once again with one student with and one without a grade, but it looks like we have found our issue:

Copy Code

```
students = [{'name': 'Alice', 'grade': 85}, {'name': 'Bob'}]
```

rest of the code omitted

```
print(average_grade(grades))
```

TypeError: unsupported operand type(s) for +: 'int' and 'NoneType'

And once again we get an error.

Now that we are getting consistent error messages and we've figured out exactly what inputs are giving us trouble, we can trace the code backward to find where the error originates.

Copy Code

```
$ python process_students.py
```

Traceback (most recent call last):

```
File "/Users/brandi/py101/lesson_2/process_students.py", line 21, in <module>
    print(average_grade(grades)) # TypeError: unsupported operand type(s) for +:
```

```
'int' and 'NoneType'
```

```
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/Users/brandi/py101/lesson_2/process_students.py", line 7, in
```

```
average_grade
```

```
    total = sum(grades)
```

```
    ^^^^^^^^^^^^^
```

```
TypeError: unsupported operand type(s) for +: 'int' and 'NoneType'
```

If we look at our stack trace, we see that we make it to invoking `average_grade` without error. That means that `collect_grades` isn't producing any errors, though it could be giving us something unexpected in our return value.

Within `average_grade`, the line that throws an error is line 7, where we invoke `sum`, passing in `grades`.

Identifying the area where an error originates is called trapping the error.

Understand the Problem Well

After we narrow the source of the bug to line 7 of the program, it's time to analyze the function's code. Let's inspect the values at each step within the function:

Copy Code

```
def average_grade(grades):
    print(grades)
    total = sum(grades)
    average = total / len(grades)
    return average
```

If we pass the original students list, we can see the output being a list `[85, None, 72, 75]`. Within the list, we can also see the problem. One of the values in the list is `None` and we can't perform addition on `NoneType`.

Video recap of this section

Implement a Fix

There are often multiple ways and multiple layers in which you can make the fix. For example, we could suppress the error from being thrown with this code:

Copy Code

```
try:
    print(average_grade(grades))
except Exception:
    print("Something went wrong")
```

We'd still have the original error in the `average_grade` function, but, depending

on where the problem lies, a solution like that may be all you can do. If you're using a library or code that you can't modify, for instance, you have little choice but to deal with edge cases in your code. Since this is our code, we can and probably should fix the offending code. While we could also fix it in the `average_grade` function, a better place is probably in the `collect_grades` function. Thus, if the student doesn't have a grade, we don't add anything to the list.

Copy Code

```
def collect_grades(students):
    grades = []
    for student in students:
        name, grade = process_student(student)
        if grade:
            grades.append(grade)
    return grades
```

Fix one problem at a time. You may notice other edge cases or problems while implementing a fix, but you should resist trying to fix them all at once. When you try to work on multiple fixes at once, it's easy to become confused and make the situation even worse than it was. Make a note of the issues you notice, finish fixing the one you're working on, then return to your list of issues and work them one at a time as well.

In case you're still getting errors, check your code against our final code -- chances are you missed an implied step or two:

Show Code

Video recap of this section

Test the Fix

Finally, after implementing a fix, ensure that your updated code fixed the problem by using a similar set of tests from step #2. After you learn about automated testing, you'll want to add an automated test to prevent regression. For now, you can test manually.

In the next assignment, we'll discuss several debugging techniques you can use with your code.

Debugging Techniques

Debugging isn't something that has any strict steps you have to follow. Every debugging problem needs to be approached independently. That said, there are several different techniques you can use. In this assignment, we'll discuss some of the more useful techniques.

Line by Line Debugging

Your best debugging tool is your patience, which is why we mentioned temperament first in this article. Most bugs in your code come from overlooking a detail, rather than a complete misunderstanding of a concept. Being careful, patient and developing a habit of reading code line-by-line, word-by-word, character-by-character is your most useful programming skill. If you are naturally impatient or like to gloss over details, you must train yourself to behave differently when programming. All other debugging tips and tools won't matter if you aren't detail oriented.

Rubber Duck Debugging

Rubber Duck Debugging sounds crazy, but it is so useful that it has its own Wikipedia page. The process centers around using some object, like a rubber duck, as a sounding board when faced with a hard problem. The idea is that when you explain the problem to a rubber duck, you force yourself to articulate the problem, detail by detail. That often leads to discovering the root of the problem. Of course, the object doesn't have to be a rubber duck -- it can be anything, including another human being. The point is to focus your mind on walking through the code, line by line, and in that process, noticing a small detail that may reveal a deeper problem. If you've never experienced this phenomenon, we encourage you to try this out yourself next time you're stuck on

a bug.

Debugging by Walking Away

Another interesting debugging technique is to go for a walk. Some have claimed a swim, jog or even a shower helps too. The idea is that this technique works since it activates a different part of your brain. When you walk away from the computer, your brain is still working on the problem in the background. Note that this is only effective if you first spend time loading the problem in your brain. You cannot just walk away when you first encounter a problem and expect this technique to work. Once you've loaded the problem in your brain, your brain may work on the problem even while you're sleeping! Also, our brains get tired too, so it's ok to step away and come back to it with fresh eyes and a refreshed brain.

Debugging with print

An often overlooked but effective debugging technique is to use print at strategic points in your code so you can inspect the state of your data. You can then check whether the data lines up with your expectations of what it should be.

Consider the following program:

Copy Code

```
def titlize(sentence):
    words = sentence.split()
    new_words = []

    for word in words:
        if len(word) > 2:
            word.capitalize()
            new_words.append(word)

    return ' '.join(new_words)
```

```
title = 'hello world of programming'
print(titlize(title))
```

This titlize function in this code is expected to return a string that represents the argument with all words of more than 2 characters capitalized. Thus, the above program should print Hello World of Programming.

However, it doesn't work. Instead, it just prints hello world programming. The words aren't capitalized, and the 2-letter word of has been stripped out. Since we aren't getting any error messages, we can't be sure where the problem lies. This is a good use case for debugging with print.

Before we start, however, we must understand the logical steps of this function. We should familiarize ourselves with what's happening to our data as it gets processed by titlize. This will help us understand when the functions actions diverge from what we expect.

As we read the code, we can see that our first job is to split the input into different words, then create a new collection to house the transformed words.

Next, we iterate over the words. If a word's length is greater than 2, we want to capitalize it. We append the word into our new_words list.

Finally, we use join to connect the words back together into a single string, separated by spaces.

There's a lot going on there, so let's start stepping through the code and confirm that we're seeing the expected results. Since we're going to be printing stuff, we probably want to remove any print calls that are already present. Thus:

Copy Code

```
def titlize(sentence):
    words = sentence.split()
    new_words = []

    for word in words:
        if len(word) > 2:
            word.capitalize()
            new_words.append(word)

    return ' '.join(new_words)
```

```
title = 'hello world of programming'
# print(titlize(title))
titlize(title)
```

Next, we want to check that `words = sentence.split()` is doing the right thing. It should set `words` to a list of the words in the input string, where the different words are based on the presence of spaces:

Copy Code

```
def titlize(sentence):
    words = sentence.split()
    print(words)
    new_words = []

    for word in words:
        if len(word) > 2:
            word.capitalize()
            new_words.append(word)

    return ' '.join(new_words)
```

```
title = 'hello world of programming'
# print(titlize(title))
titlize(title)
```

If we run this code, we should see the following output:

OutputCopy Code

```
['hello', 'world', 'of', 'programming']
```

That's what we expect, and that's what we get. Therefore, we know that the function is working correctly up to the print invocation.

A logical next step is to check whether our loop is properly iterating over all the words in the sentence:

Copy Code

```
def titlize(sentence):
    words = sentence.split()
    # print(words)          # line deleted
    new_words = []

    for word in words:
        print(word)
        if len(word) > 2:
            word.capitalize()
            new_words.append(word)

    return ' '.join(new_words)
```

```
title = 'hello world of programming'
# print(titlize(title))
titlize(title)
```

This time, we expect to see the program print the following:

OutputCopy Code

```
hello
world
of
```

programming

When we run the code, that's exactly what gets printed, so the loop itself is working.

Let's check the if statement by adding a print statement inside the if's body:

Copy Code

```
def titlize(sentence):
    words = sentence.split()
    new_words = []

    for word in words:
        # print(word)      # line deleted
        if len(word) > 2:
            print(word)
            word.capitalize()
            new_words.append(word)

    return ' '.join(new_words)
```

```
title = 'hello world of programming'
```

```
# print(titlize(title))
```

```
titlize(title)
```

At this point, we're expecting to see the words that have a length of more than 2:

OutputCopy Code

```
hello
world
```

programming

Once again, that's what we get. That tells us that the if condition is working properly.

Next, we want to check whether the capitalization works:

Copy Code

```
def titlize(sentence):
    words = sentence.split()
    new_words = []

    for word in words:
        if len(word) > 2:
            # print(word)      # line deleted
            word.capitalize()
            print(word)
            new_words.append(word)

    return ' '.join(new_words)
```

```
title = 'hello world of programming'
```

```
# print(titlize(title))
```

```
titlize(title)
```

This time, we expect to see the longer words properly capitalized:

Expected OutputCopy Code

```
Hello
World
```

Programming

However, that isn't what we get when we run the code:

Actual OutputCopy Code

hello

world

programming

Since this isn't what we expected, we know that something has gone wrong, most likely with `word.capitalize()`.

That is, indeed, where this problem lies. Strings are immutable, so a method call like `word.capitalize()` won't change the value of `word`. Instead, we need to capture the return value and reassign `word` to that value:

Copy Code

```
def titlize(sentence):
    words = sentence.split()
    new_words = []

    for word in words:
        if len(word) > 2:
            word = word.capitalize()
            print(word)
            new_words.append(word)

    return ' '.join(new_words)
```

```
title = 'hello world of programming'
```

```
# print(titlize(title))
```

```
titlize(title)
```

When we run this updated code, we now get the expected result showing capitalized words:

OutputCopy Code

Hello

World

Programming

We're not done yet, however. If we restore the original `print` invocation and get rid of the debugging `print`, we can see that we're still missing the word of:

Copy Code

```
def titlize(sentence):
    words = sentence.split()
    new_words = []

    for word in words:
        if len(word) > 2:
            word = word.capitalize()
            # print(word)          # Delete line
            new_words.append(word)

    return ' '.join(new_words)
```

```
title = 'hello world of programming'
```

```
print(titlize(title))
```

```
# titlize(title)
```

OutputCopy Code

Hello World Programming

So, what's next? Try to figure out where the remaining bug is, and fix it. For practice, try to debug the code with `print`; don't try to do it by just studying the code. The solution is included in the video walkthrough for this section:

Inspecting with a Debugger

Python comes with a built-in debugger called `pdb`, which lets you pause the program during execution and inspect values at runtime. Let's walk through how to use `pdb` with an example. We'll start with some simple working code to gain familiarity with the basics of `pdb`, after which we'll look at code that needs

debugging. (We also have a video at the end of this section.)

Note for Python versions 3.13 and later:

Starting in Python 3.13, `pdb.set_trace()` will pause the program on the line where it is called, rather than on the next line of code to be executed. If you're using Python version 3.13 or later, your output and behavior will be slightly different from what you see in these examples and in the video walkthrough.

If you use the next command after starting the debugger, you'll be paused in the correct place to follow along with the examples.

To use the debugger, you need to import the `pdb` module and use the `pdb.set_trace()` function to set a breakpoint in your code.

Copy Code
debug.py

```
import pdb

counter = 1

while counter <= 5:
    print(counter)
    pdb.set_trace() # Add breakpoint
    counter += 1
```

Save this code in a file called `debug.py` and run it in the terminal:

Copy Code
\$ python debug.py
You will see a prompt with some output like the following:

OutputCopy Code
1
> /Users/xyzzzy/ls_test/test.py(10)<module>()
-> counter += 1
(Pdb)
The line `-> counter += 1` means that the program execution stopped at this line, which is where we added our breakpoint.

Note that we can add multiple breakpoints if needed:

Copy Code
debug.py

```
import pdb

counter = 1

pdb.set_trace() # Another breakpoint

while counter <= 5:
    print(counter)
    pdb.set_trace() # Add breakpoint
    counter += 1
```

We can access the value of any variables in scope by using the `p` command followed by the variable name, for example, `p counter`.

Copy Code
1
> /Users/xyzzzy/ls_test/test.py(10)<module>()
-> counter += 1
(Pdb) p counter # Our command


```
1          # The output, the current value of `counter`  
To continue execution to the next breakpoint, you can use the c command (short  
for "continue"). This will execute the code until the next breakpoint is  
encountered. If we run p counter after we've invoked the c command once, we  
should get the expected value, 2, as counter += 1 has been executed once.
```

You can continue using the c command to pause at the breakpoint for each iteration of the loop.

If you just wanted to move on to the next line of the code we would use the n command instead.

Finally, you can exit the debugger by typing q and pressing Enter. In the following video, we walk through this example and then work through a trickier code snippet that requires debugging.

Video Walkthrough with pdb

How is setting a breakpoint different from using print?

[Click here to ask LSBot this question](#)

Type your answer to the question above... (Press Cmd/Ctrl+Enter to send)

[Check My Answer](#)

Advanced Debugging

The pdb debugger has a lot of other helpful features. For example, it has mechanisms to print all arguments with their values in the current function etc. We won't cover that yet. For now, we just want to introduce the concept of a debugger and cover some of the basic usage. This should be sufficient to help debug small programs.

Summary

In summary, debugging is arguably the most vital skill you must learn as a programmer. Focus on developing a patient, systematic temperament; carefully read error messages; use all the resources at your disposal. You should approach debugging in sequential steps and use the techniques we covered above -- especially the debugger.

Errors

Python's flexibility lets a developer use it in many ways, but it also lets you easily write code that fails to function when run. There are many situations where the Python interpreter cannot continue executing a program. Instead, it creates an Exception object that describes the problem and stops the program.

As a developer, you want to do your best to avoid errors when your programs run. Let's look at the most common types of errors that can arise. In the next few assignments, we'll learn techniques to both prevent errors and handle those that you can't avoid.

Terminology

When an error occurs in a Python program, we say that it raises an Exception. Some programming languages use the term errors to denote exception; some other languages refer to throwing exceptions. Thus, you might hear fellow developers talk about exceptions being thrown in Python programs. Despite these terminological differences, remember that the terms Exception and Error are interchangeable, just as raise and throw are.

NameError

A NameError arises when you attempt to use a variable or function that hasn't been defined.

Copy Code

```
a          # Referencing an undefined variable results in a NameError.  
a()        # The same applies when trying to call an undefined function.  
TypeError
```

A `TypeError` occurs when a value of the wrong type is used in an expression. For instance, the following situations should raise a `TypeError` exception:

using an argument of the wrong type as a function argument

Copy Code

```
>>> my_str = "abc"
>>> my_str.find(42) # TypeError: must be str, not int
trying to call an object that isn't callable:
```

Copy Code

```
>>> my_int = 42
>>> my_int() # TypeError: 'int' object is not callable
```

SyntaxError

A `SyntaxError` occurs when Python encounters code that does not meet its syntactic rules. For instance, a string that lacks the trailing quote character is a syntax error:

Copy Code

```
print('hello)
# SyntaxError: unterminated string
A SyntaxError is unique in that it typically arises immediately after loading a Python program but before it starts running. Unlike NameError and TypeError, which hinge on specific variables and values encountered during runtime, Python detects SyntaxErrors solely from the program's text.
```

Copy Code

```
def ( # SyntaxError: unexpected EOF while parsing
Several cases can lead to a SyntaxError while a Python program is running. For example, this code would result in a SyntaxError at runtime:
```

Copy Code

```
expression = "2 * (3 + 4"
result = eval(expression)
That code uses Python's eval function to evaluate an Python expression that has been written as a string. Thus, the above code is effectively the same as:
```

Copy Code

```
result = 2 * (3 + 4
The key difference is that the eval function raises a SyntaxError when the code is executed, but the second code fragment raises a SyntaxError during the parsing phase.
```

Don't worry about remembering what `eval` does. While it has its uses, it's a function that is best avoided by most developers. It's slow and potentially very dangerous from a security standpoint.

ValueError

A `ValueError` is raised when a function receives an argument of the correct data type, but the value of the argument is inappropriate for the operation.

Copy Code

```
number = int("abc")
# ValueError: invalid literal for int() with base 10: 'abc'
```

IndexError

An `IndexError` occurs when trying to access an index of a sequence (like a list or string) that is outside the range of valid indexes.

Copy Code

```
nums = [1,2]
num = nums[2] # IndexError: list index out of range
```

KeyError

A `KeyError` is raised when trying to access a dictionary key that doesn't exist.

Copy Code

```
my_dict = {"a": 1, "b": 2}
value = my_dict["c"] # KeyError: 'c'
ZeroDivisionError
```

A ZeroDivisionError occurs when attempting to divide by zero or when trying to use 0 on the right side of the modulo (%) operator:

Copy Code

```
result1 = 10 / 0 # ZeroDivisionError: division by zero
result2 = 42 % 0 # ZeroDivisionError: integer modulo by zero
Exception Handling
```

Exception handling lets you gracefully manage errors that might occur during program execution. Python provides a structured way to handle exceptions using the try, except, else, and finally statements.

Here's how the exception handling process works:

Try Block: The code that might raise an exception is placed within the try block. Python will monitor this block for any exceptions that may occur during its execution.

Except Block: If an exception is raised in the try block, Python will look for a matching except block that can handle that specific type of exception. If a match is found, the code within the corresponding except block is executed.

Else Block (Optional): The else block is executed only if no exceptions occurred in the try block. It's used for code that should run when no errors were encountered.

Finally Block (Optional): The finally block is always executed, regardless of whether an exception was raised or not. It is used for cleanup operations or tasks that must be performed, such as releasing resources.

Let's walk through an example that demonstrates the different aspects of exception handling in Python:

Copy Code

```
try:
    num_str = input("Enter a number: ")
    num = int(num_str)

    result = 10 / num
except ValueError:
    print("Invalid input, you didn't enter a number.")
except ZeroDivisionError:
    print("Cannot divide by zero.")
else:
    print(f"Result: {result}")
finally:
    print("Exception handling complete.")
```

In this example:

If the user enters a non-integer value, a ValueError is caught, and the program displays an error message.

If the user enters zero, a ZeroDivisionError is caught, and the program warns about division by zero.

If no exceptions occur, the division result is printed.

Regardless of whether an exception was caught or not, the finally block ensures that the final message is displayed.

Preventing Errors

In Python programming, preventing errors and handling exceptions is crucial for writing reliable code. There are two main approaches to error prevention: "Look Before You Leap (LBLY)" and "It's Easier to Ask Forgiveness than Permission

(EAFP)." This assignment will cover both approaches.

LBYL: Look Before You Leap

The LBYL approach involves checking for potential errors before executing code that might fail. Here's an example of LBYL code:

Copy Code

```
def lower_first(word):
    # Ensure word is a string
    if type(word) != str:
        return word

    # Ensure word contains at least one character
    if len(word) == 0:
        return word

    # We now know that word is a string that contains
    # at least one character. That means the following
    # code will run without generating an error.
    return word[0].lower() + word[1:]
```

```
print(lower_first("F00")) # Output: "f00"
```

```
print(lower_first(32))    # Output: 32
```

Guard Clauses

The LBYL approach typically uses one or more guard clauses to ensure that data meets the specific preconditions a function expects. In the `lower_first` function above, we have two guard clauses:

The guard clause on lines 3 and 4 ensures that `word` is a string. If it is not, we just return the string as-is.

The guard clause on lines 7 and 8 ensures that `word` has at least one character. If it does not, we just return the string as-is (it's empty).

When To Use Guard Clauses

Guard clauses are best used when a function cannot assume that its arguments are valid. Invalid arguments can have incorrect types, structures, values, or properties. However, it's typically ideal for your program to be confident in its ability to use valid values. For instance, if you can trust that your program always calls `lower_first` with a non-empty string, you might not need the guard clause.

EAFP: It's Easier to Ask Forgiveness than Permission

The EAFP approach involves trying an operation and handling any exceptions that arise. This approach assumes that the code will execute successfully and handles exceptions only if something goes wrong. Here's an example of EAFP code:

Copy Code

```
def lower_first(word):
    try:
        return word[0].lower() + word[1:]
    except (TypeError, IndexError):
        return word # Handle exceptions by returning `word` as-is
```

```
print(lower_first("F00")) # Output: "f00"
```

```
print(lower_first(32))    # Output: "32"
```

Exception handling often arises in EAFP code due to its nature of trying operations without explicit checks. We have discussed exception handling in the Errors assignment so make sure to review it if necessary.

The author of Python is agnostic about the best approach, suggesting that you use whichever approach suits your specific situation. However, in the Python community, the EAFP approach is generally preferred for its readability and concise code.

Detecting Edge Cases

Much of error prevention involves examining the underlying assumptions in your code. Consider whether your program can challenge those assumptions. These situations are often called edge cases since they tend to involve values at the extreme edges of the potential value range. In `lower_first`, the shortest possible string ('') is an edge case.

To identify edge cases that could potentially disrupt your program, begin by analyzing the inputs to your code. In most functions, these inputs are typically the arguments. Each argument may have certain values that can lead to unexpected behavior.

For instance, consider an argument that should be numeric. Will your code still function properly if the argument is negative or zero? Imagine you're anticipating a whole number (integer), but instead receive a value with a fractional component. These are common situations where a valid numeric object can defy expectations, potentially causing unintended behavior and bugs.

In the case of `lower_first`, we observed that empty strings can cause issues when the function isn't prepared for them. Numerous other types of strings can also pose challenges, such as those beginning or ending with spaces, consisting solely of spaces, or containing special characters.

Lastly, consider contemplating how specific combinations of values can lead to unforeseen conditions.

Planning Your Code

While you can't account for every possible permutation of arguments, it does pay to plan ahead. One good way to do this is to write out the common use cases of a new function, and check how your function handles them. This is a great way to identify edge cases.

Let's say we are writing a function that inserts a new element to a list in its proper alphabetically sorted position:

Copy Code

```
countries = ['Australia', 'Cuba', 'Senegal']
```

```
alpha_insert(countries, 'Brazil')
```

```
print(', '.join(countries)) # Outputs "Australia, Brazil, Cuba, Senegal"
```

Let's think about some use cases; we want to ensure that `alpha_insert` can handle:

Copy Code

```
alpha_insert([], 'Brazil') # Inserting into an empty list
```

```
alpha_insert(['Brazil'], 'Australia') # At the beginning of a list
```

```
alpha_insert(['Brazil'], 'Cuba') # At the end of a list
```

```
alpha_insert(['Brazil'], 'Brazil') # Duplicate entry
```

While we have only four main cases here, more complex functions can involve numerous cases. Handling them all might become overwhelming. Instead, focus on the "happy path" -- the most basic use case(s). Afterward, revisit your comprehensive list of use cases and ensure your implementation functions effectively in each scenario. If a specific case fails, address it and then reevaluate the use cases.

Note that our list of use cases ignores the problem of invalid data types passed as arguments (for instance, passing a `Number` when a function expects a `String`). You can check argument types when this is a real possibility, but doing so in every function is unneeded and difficult to maintain.

Precedence

What do you think `3 + 5 * 7` returns? Which operation gets evaluated first - `7 * 5` or `5 + 3`? If `7 * 5`, then we get a return value of 38. Otherwise, we get 56. It turns out that `7 * 5` gets computed first. But why?

The meaning of an expression in Python is determined by what is called operator precedence. It's a set of rules that dictate how Python determines what operands each operator takes. Operands are simply values -- the results of evaluating expressions -- that are used by the operator.

For most operators, there are two operands; however, there is also a "unary" operator that takes one operand.

Copy Code

```
2 + 5          # Two operands (2 and 5)
not True       # Unary: One operand (True)
```

Each operand can be the result of evaluating some other operator and its operands. Consider this expression:

Copy Code

```
3 + 5 * 7  # Result: 38
```

Here we have 2 operators (+ and *), each of which takes 2 numbers as operands. However, there are only 3 numbers here. What gives? Furthermore, what does this expression evaluate to?

The answer lies in how operator precedence works. Precedence determines whether the + operator uses 3 and 5 as operands or 3 and 35 (5 * 7). Likewise, it determines whether * uses 5 and 7 as operands or 8 (3 + 5) and 7. In short, precedence determines the meaning of an expression.

In an expression, operators with higher precedence are prioritized over those with lower precedence. In 3 + 5 * 7, * has higher precedence than +, so * gets passed 5 and 7 as operands, which returns 35 as a result. Subsequently, 3 and 35 get passed to + for a final result of 38. It's as though Python inserted a pair of parentheses around 5 * 7:

Copy Code

```
3 + (5 * 7)  # Result: 38
```

Suppose we wanted to prioritize 3 + 5 instead of 5 * 7? The solution is as simple as using our own set of parentheses:

Copy Code

```
(3 + 5) * 7  # Result: 56
```

Parentheses override the default evaluation order and have the highest possible precedence.

Another way to think of precedence is that it controls the order of evaluation. Operations involving operators with high precedence get evaluated before operations involving low precedence. When two operations involve operators of the same precedence, the operations occur left-to-right (or right-to-left in some cases). However, thinking of precedence in this way can sometimes lead to unexpected results when using the or and and short-circuit operators. It's safer to view precedence as the mechanism that determines which operands are passed to each operator.

While operator precedence is a valuable tool, it's not always foolproof. Relying solely on precedence might lead to confusion and unexpected results. When dealing with expressions involving two or more different operators, using parentheses to explicitly define the intended order is recommended. Additionally, even the left-to-right evaluation can be prone to issues, such as when dealing with multiple assignments or exponentiation (**).

An operator that has higher precedence than another is said to bind more tightly to its operands. In the expression 3 + 5 * 7, the * operator binds more tightly to its operands, 5 and 7, than does the + operator. Thus, + binds to 3 and the return value of 5 * 7 instead of 3 and 5.

Evaluation Order

From time to time, you may hear or read somebody saying that precedence determines the order in which expressions get evaluated. The evaluation process is more complicated than just determining what gets evaluated first, though. In fact, precedence in Python is only part of the story; the other parts are either left-to-right evaluation, right-to-left evaluation and short-circuiting.

Consider this code and the output it displays:

Copy Code

```
def value(n):  
    print(n)  
    return n
```

```
print(value(3) + value(5) * value(7))
```

Copy Code

```
3  
5  
7  
38
```

From the first 3 lines of output, you might conclude that Python is evaluating the expression left-to-right. However, the final result says otherwise -- you can only get that result if `value(5) * value(7)` gets evaluated before `value(3) + value(5)`. Which is it?

It's a little of both. The issue here is that operators like `+` and `*` need values that they can work with. Function invocations like `value(5)` and `value(7)` are not values. We can't invoke the `*` operator until we know what values those functions return. In an arithmetic expression, Python first goes through an expression left-to-right and evaluates everything it can without calling any operators. Thus, here it evaluates `value(3)`, `value(5)`, and `value(7)` first, in that order. Python re-evaluates the result with the precedence rules only after it has those values.

Right-to-left evaluation occurs when you do multiple assignments or multiple `**` operators:

Copy Code

```
a = b = c = 3  
5 ** 3 ** 2 # 1953125 (same as 5 ** (3 ** 2) = 5 ** 9 = 5 * 5 * 5 * 5 * 5 * 5 *  
5 * 5 * 5)
```

Neither of these are good practice in Python, so we won't discuss them further.

The short-circuit operators `and` and `or` can lead to unexpected behavior regarding precedence. Consider these expressions:

Copy Code

```
5 and 1 / 0 # ZeroDivisionError  
None or 1 / 0 # ZeroDivisionError  
What happens, though, if we modify things so that 1 / 0 isn't needed?
```

Copy Code

```
None and 1 / 0 # None  
5 or 1 / 0 # 5
```

In both cases, `1 / 0` never gets executed, even though operator precedence would suggest that it should be evaluated first. However, these two expressions don't evaluate `1 / 0` due to short-circuiting. This is simply the way Python works - it treats `and`, `or` differently from other operators and doesn't evaluate subexpressions unless it needs them.

Is every part of an expression guaranteed to be evaluated? If not, explain why and give an example.

[Click here to ask LSBot this question](#)

Type your answer to the question above... (Press Cmd/Ctrl+Enter to send)

Check My Answer

Use Parentheses!

Really. Don't rely on the precedence rules when you're mixing operators; use parentheses whenever you mix operators in an expression. If you don't bother with the parentheses, you'll eventually decide to work from memory. Sooner or later you'll misremember the rule, and introduce a nice little bug into your code. Someone reading your code may not be as familiar with the precedence rules as you. They will either have to look up the rules, or assume that you knew what you were doing when you wrote the code. If they assume that you knew what you were doing, they will miss the bug.

Type Conversions

Type coercion is the process of converting a value of one type to another. While all programming languages have mechanisms for coercion, type coercion in Python is generally straightforward compared to some other languages. In this assignment, we'll explore how type coercion works in Python and discuss various ways to coerce values between different types.

Perhaps the most common case of type coercion is when a programmer employs various built-in functions to convert a value of one type to another. In our earlier calculator program, we used the `int` and `float` functions to convert strings into numerical values before performing arithmetic operations. This form of type coercion using `int` and `float` functions is quite prevalent in Python programs. The need to convert strings to numbers frequently arises, especially when dealing with user inputs, which are often provided as strings.

Coercing values to Integers

The `int()` function is used to convert values to integers. It takes a value as input and attempts to convert it to an integer.

Copy Code

```
integer_str = "42"
integer = int(integer_str)
print(integer) # Output: 42
```

Attempting to convert a non-numeric string to an integer using `int()` will raise a `ValueError`.

Copy Code

```
non_numeric_str = "abc"
try:
    non_numeric_integer = int(non_numeric_str)
except ValueError:
    print("Cannot convert to int")
```

Most often, we pass a string value to the `int()` function. However, it can also accept a real number (floating-point numbers), bytes-like object (which we won't discuss in this assignment), and, surprisingly, it also won't raise an error if we pass a boolean to it.

Passing any other data type, like a list for example, will result in a `TypeError`.

Copy Code

```
input_value = ['42']
try:
    numeric_value = int(input_value)
except TypeError:
    print("Invalid input type for conversion")
```

Coercing values to Floats

The `float()` function is used to convert values to floating-point numbers. It works similarly to `int()`.

Copy Code

```
float_str = "3.14"
float_number = float(float_str)
```



```
print(float_number) # Output: 3.14
```

Like with `int()`, attempting to convert a non-numeric string to a float using `float()` will raise a `ValueError`.

And also, just like the `int()` function, if we pass any other data type, other than string, a real number, bytes-type object, and a boolean to it, it will raise a `TypeError`.

Copy Code

```
input_value = {'value': 42}
```

```
try:
```

```
    result = float(input_value)
```

```
except TypeError:
```

```
    print("Invalid input type for conversion")
```

Floats have a special "Not-a-Number" value `nan`. Not-a-Number is one of the common ways to represent the missing value in the data. It typically arises from operations that don't have a meaningful result. For example:

Copy Code

```
result = float('inf') - float('inf')
```

```
print(result) # nan
```

In this example, we subtract positive infinity from positive infinity, which results in a `nan` value.

When coercing values to floats using the `float()` function, it's important to note how `NaN` is handled:

Copy Code

```
nan_string = "NaN"
```

```
nan_float = float(nan_string)
```

```
print(nan_float) # Output: nan
```

Coercing values to Strings

The `str()` function is used to convert values to strings. It is a versatile function that works with all built-in Python data types as well as most non-built-in types, providing a consistent way to represent values as strings.

Copy Code

```
str(True) # 'True'
```

When you use the `print()` function to output a value, Python automatically converts the value to a string using the `str()` function. This makes it convenient to display variables and other values in a human-readable format without needing to explicitly call `str()`.

Copy Code

```
number = 42
```

```
print("The answer is:", number) # Output: The answer is: 42
```

String interpolation is a common technique for including values within a string. When you use interpolation, Python automatically coerces the values to strings using the `str()` function.

Copy Code

```
name = "Karl"
```

```
age = 30
```

```
message = f"Hello, my name is {name} and I am {age} years old."
```

```
print(message) # Output: Hello, my name is Karl and I am 30 years old.
```

Coercing values to Booleans

The `bool()` function is used to convert values to booleans. It works with all built-in Python values and most non-built-in values. When using the `bool()`

function, it returns `True` if the value is truthy (evaluates to `True` in a boolean context), and `False` if the value is falsy (evaluates to `False` in a boolean context).

Copy Code

```
truthy_value = "Hello"
falsy_value = None
```

```
is_truthy = bool(truthy_value)
is_falsy = bool(falsy_value)
```

```
print(is_truthy) # Output: True
print(is_falsy) # Output: False
```

In the example above, the `truthy_value` is converted to `True` since it is a non-empty string, while the `falsy_value` is converted to `False` since it is `None`.

`repr()` vs `str()`

In Python, the `repr()` function returns a string representation of an object, which can be used to recreate the object using Python code.

Copy Code

```
x = [1, 2, 3]
repr_x = repr(x)
```

```
print(repr_x) # Output: [1, 2, 3]
```

While the `str()` function can be used to convert objects to strings, these two functions serve a slightly different purpose:

`str()`: This function is used to create a string representation of an object that is meant to be human-readable. It focuses on providing a concise and user-friendly representation of the object's value. It is often used for display purposes, such as in `print()` and string interpolation.

`repr()`: This function is used to create an unambiguous string representation of an object that can be used to recreate the object. It is often used for debugging and development purposes. The `repr()` representation should ideally be a valid Python expression that, when evaluated, would recreate the original object.

Let's see the difference:

Copy Code

```
import datetime
```

```
today = datetime.datetime.now()
```

```
print(str(today)) # 2023-08-10 10:18:05.535262
```

```
print(repr(today)) # datetime.datetime(2023, 8, 10, 10, 18, 5, 535262)
```

As you can see from the example above, the `str()` function shows the date and time in a format that is easy for humans to understand.

On the other hand, the `repr()` function prints the developer-friendly representation of the date object. This representation can be directly used in Python code to recreate the same `datetime` object.

If you need a value to be a specific type but aren't sure what type it is currently, what are two strategies you could use to convert or handle the value safely?

[Click here to ask LSBot this question](#)

Type your answer to the question above... (Press `Cmd/Ctrl+Enter` to send)

Check My Answer

Combining Integer with Float

In Python, when you conduct a calculation involving both an integer and a float, the system will automatically adjust the integer to a float, ensuring the outcome is a float as well. This type of coercion is sometimes referred to as implicit type coercion.

Copy Code

```
x = 3 # Integer
```

```
y = 2.0 # Float
```

```
result = x + y
```

```
print(result) # Outputs: 5.0
print(type(result)) # Outputs: <class 'float'>
```

Here, although `x` is an integer, the result is a float because of the presence of the float `y` in the arithmetic operation.

Combining String with Non-string

Unlike many other languages, Python is not real big on coercing non-strings to strings. Thus, if you try to use the `+` operator with a string and a number, Python will raise a `TypeError`.

Copy Code

```
name = "Clare"
age = 35
```

```
print(name + age)
```

```
# TypeError: can only concatenate str (not "int") to str
```

One place where non-strings are automatically converted to string is in the `print()` function. `print()` will implicitly convert any non-string arguments to a string before printing them.

Copy Code

```
name = "Clare"
age = 35
```

```
print(name, age) # Outputs: Clare 35
```

Coding Tips

We perform many code reviews, so we see a wide variety of code. We want to mention a few specific items we've seen come up over and over in doing reviews that may help you on your journey.

Dramatic Experience and Retaining Knowledge

A Launch School graduate was talking about how far he's come since starting from the first course and how much he's learned and how much has stuck with him. However, there were also some mistakes that he continues to make, despite having learned about them early on.

After chatting a bit, we concluded that he was still making those basic mistakes since he hadn't been burned badly enough. On the flip side, the things he always remembered were usually due to problems that caused him a major headache in the past. Any time those issues came up, the burn was fresh in his mind, and that helped him avoid repeating the mistake.

Part of our goal at Launch School is to help people realize that programming isn't all that hard. Unfortunately, it does require repetition. Maybe it takes you 2 to 3 hours to debug a small problem, and then you realize "Oh, I just missed a comma" - that's your burn.

When you first start learning to program, it seems like there are too many rules to remember. It's hard to understand which ones are crucial, and which ones aren't worth memorizing. Most of the time, when you come across yet another suggestion to follow, the reaction is "Yea, OK! Another rule I need to remember. I'll file it away... somewhere." Until you burn a couple of hours on that rule, you won't retain it for the long haul.

The only way to retain information is to pay with time. Debugging an issue for hours and hours ensures that the problem gets burned into long-term memory. You pay for those burns with time, but they pay you back with interest. If you spend 3 hours debugging, those hours weren't wasted -- you won't make that mistake again. Think about debugging from that perspective -- embrace your burns and remember their lessons.

If you are serious about programming and you want to do it for years and maybe decades from today, then the hours you put into debugging little things are

going to help you retain knowledge for the long haul.

You're more likely to remember things that took you a long time to solve or that caused you some embarrassment. You decide how to pay for the experience, either through time or repetition.

Naming Things

Many people try to save on characters by using very short variable or function names. There's no need to save on characters. Choose descriptive names. For example, we see code like this often:

Copy Code

```
p = get_user_input()
```

What is `p`? It's easy to see that we're collecting some user input here, but what, exactly, does `p` contain? The next time we use `p` may be much later in the program. By then, you may no longer remember what it contains. That's especially true when you return to a program months or years after you wrote it.

Variable names should describe the content of the variable. If the name doesn't reflect that information, it adds another mental check that you must perform every time you see `p`. We could name it `yes_or_no`, but another problem arises if you later decide to allow a "maybe" response from the user. Typically, you don't want to hardcode possible values into a variable name because of future uncertainty. Instead, try to capture the intent of the variable. For example, if we're trying to capture a response to determine whether the user wants to play a game again, a better name would be `play_again`. It's both descriptive and future-proof.

Naming Conventions

Names that follow the naming conventions in the Python style guide are referred to as idiomatic names. In particular, whether a name is idiomatic or not depends on what kind of name we're describing. The following tables show which names are and aren't idiomatic in the various categories (we'll talk about constructors and classes later) and when.

Category Name Notes

Non-constant variables and object properties `employee`

`number`

`fizz_buzz`

`speed_of_light`

`destination_url` `url` is an acronym

`_var` meant for internal use

`var_` used to avoid conflicts with keywords

Classes `Cat`

`BoxTurtle`

`FlightlessBird`

Functions and methods `parse_url` `url` is an acronym

`go_faster`

Constant Names `DEFAULT_TIMEOUT` `SCREAMING_SNAKE_CASE`

`MIN_AGE`

Valid but Non-Idiomatic Names

Category Name Notes

Universally non-idiomatic `İ` Non ASCII characters

`__var__` except as defined by Python

`user_Profile` should be all lowercase

`UserProfile` camelCase is not idiomatic

`milesperhour` Undifferentiated words

Non-constant variables and dictionary keys `FIZZ_BUZZ` `SCREAMING_SNAKE_CASE`

`fizzBUZZ` camelCase (sorta); non-idiomatic

Classes `cat` Begins with lowercase letter

`make_turtle` `snake_case`

Invalid Names

Name Notes

`42ndStreet` Begins with a digit

```
fizz buzz    contains space
fizz-buzz    contains special character except underscore
for Python keyword
```

Avoid Magic Numbers

A magic number is a number (or other simple value) that appears in your program without any information that describes what that number represents. For instance, a card game in which each player is dealt 5 cards may use the number 5 at various points in the program, such as a loop that deals the cards or when determining how many cards a player should receive to get back to the 5-card level. If you just use the number 5 in your program, there's no way to tell by just looking at the code why you are using that particular number. The situation becomes even more confusing when you use the same number in other contexts. For instance, your game may also use a best-of-five approach to determine the overall winner.

The way to avoid magic numbers is to use constants.

Copy Code

```
NUMBER_CARDS_IN_HAND = 5
```

```
def deal_hand():
    hand = []
    for card_number in range(NUMBER_CARDS_IN_HAND):
        hand.append(deal_card())
```

```
    return hand
```

Typically, magic number constants are set at the top level of a program as shown above, though you can also define them inside a function.

Another consideration when defining constants is whether the meaning of the number is clear. For instance, in some programs, you may need to know the Unicode code points (sometimes erroneously referred to as the ASCII character-code values) for the characters a and z. You could define those constants like this:

Copy Code

```
FIRST_CHARACTER_CODE = 97
LAST_CHARACTER_CODE = 122
```

This code works, but it's still not clear what 97 and 122 mean in the context of your program. Instead, you can use some more explicit code by calculating the values directly from the characters:

Copy Code

```
FIRST_CHARACTER_CODE = ord('a')
LAST_CHARACTER_CODE = ord('z')
```

In this last code fragment, `ord('a')` returns the Unicode code point of the character a, while `ord('z')` does the same thing for z. The resulting definition is now much clearer and also less error-prone.

Formatting

When indenting code, we will always use four space characters. We never use tab characters.

When using indentation to define code blocks, you should also be mindful of the line length to ensure the code remains easy to read and understand.

Copy Code

```
def calculate_sum(a, b):
    return a + b
```

Insert spaces around operators to improve code readability. This helps in separating operators from their operands and makes the code easier to scan.

Copy Code

```
total = num1 + num2 # Good
```

```
total=num1+num2      # Avoid
```

Mutating Constants

Constants are values that remain unchanged throughout the program's execution. In Python, you should avoid mutating (changing) constants to ensure the integrity of your code's logic. Use `SCREAMING_SNAKE_CASE` to indicate constants, and treat them as read-only values. Consider the following code:

Copy Code

```
CARDS = [1, 2, 3]
```

```
CARDS.append(4)
```

```
print(CARDS) # [1, 2, 3, 4]
```

```
CARDS[1] = 'changed'
```

```
print(CARDS) # [1, 'changed', 3, 4]
```

```
CARDS.pop(0)
```

```
print(CARDS) # ['changed', 3, 4]
```

Here, we add a new element to the list object given by `CARDS`, replace an element with a new value, and remove an element from the list. All 3 operations are honored by Python since they merely mutate the list.

At best, mutating a value that is assigned to a constant is misleading and should be avoided. However, it's not illegal to do so and not always observed in practice.

Function Guidelines

The instinct to extract code to a function is good. However, make sure that the function does one thing, and that its responsibility is limited. That implies that your functions should be short (say, 10 lines or less). If it's more than 15 lines long, consider splitting it into 2 or more functions.

A function is said to have side effects if it does any of the following:

It reassigns any non-local variable. Reassigning a variable in the outer scope would be a side effect.

It modifies the value of any data structure passed as an argument, or accessed directly from the outer scope. Mutating an object, such as appending an element to a list argument, is an example of a side effect.

It reads from or writes to a file, network connection, browser, or the system hardware. Side effects like this include printing and reading input from the terminal.

It raises an exception without handling it.

It calls another function that has side effects.

The following functions have side effects:

Copy Code

```
# side effect: prints output
```

```
# returns: None
```

```
def display_total(num1, num2):  
    print(num1 + num2)
```

```
# side effect: mutates the passed-in list
```

```
# returns: updated list
```

```
def add_to_list(target_list, value_to_append):  
    target_list.append(value_to_append)  
    return target_list
```

Now, here's an example of a function with no side effects:

Copy Code

```
# side effect: none
# returns: a new number
```

```
def compute_total(num1, num2):
    return num1 + num2
```

Most functions should return a useful value or they should have a side effect, but not both. In the above examples, `add_to_list` both returns a useful value and has a side effect. If you write functions that do both, you may have trouble remembering one of those -- either you'll forget about the side effect, or you'll forget that there's a return value that you need to examine.

By "useful value," we mean that the function returns a value that has meaning to the calling code. For instance, a `compute_total` function should probably return a number that contains the result of adding some numbers together. A function that returns an arbitrary value or that always returns the same value (such as `None`) is not usually returning a useful value.

There are exceptions to this rule about mixing side effects and useful return values. For instance, if you read something from a database, you almost certainly have to return a value. If you read some input from the user's keyboard, you probably need to display a prompt, read the input from the terminal, then return a value. Accessing a database and reading and writing from the terminal are side effects, but you may still need a return value.

Function names should reflect whether side effects may occur. For instance, you can use a name like `display_total` as the name of a function that displays a total on the console. The term "display" implies that you're going to do some output -- a side effect -- rather than attempting to just calculate and return the total. On the other hand, you would probably name the function that computes the total something like `compute_total`. In this case, "compute" implies that you're going to return the value of a computation.

Functions should be at the same level of abstraction

This guideline is a little hard in the beginning since you have to develop a feel for it over time.

Usually, functions take some input and return an output. We should be able to copy and paste the function into the REPL and test it. When working with a function, you should mentally extract the function from the program and work with it in isolation. You should be able to feed it inputs and expect it to produce some outputs. When you have a function like that, you can use it without thinking about the implementation. Working this way helps compartmentalize your focus, an important skill when working on large code bases.

If the functions in your program are correctly compartmentalized, it makes programming much simpler, especially on large programs. You can see this when you read good code; the functions are all at the same layer of abstraction.

For example, given the four functions below, which one stands out?

```
deal()
hit()
stay()
iterate_through_cards()
```

The last one, `iterate_through_cards`, is not at the same abstraction level as the other functions. The other functions are in the language of the game -- verbs that are used only for this game. They all specify "what" to do, but leave the implementation details to the actual function. You shouldn't care about the implementation when you use the function. The last function is a programmer concern -- iterating through cards. It's "how" you perform the task.

Again, this is going to be hard in the beginning, but pay attention to how you organize your functions, and whether you can look at a list of functions a week later and still understand how to use them without studying their implementations.

Function Names Should Reflect What They Do

Function names should reflect whether side effects may occur:

Copy Code

```
def updated_total(total, cards):  
    # ... some code here
```

When we see a function called `updated_total`, we assume that it mutates something -- perhaps one of the arguments or something else in the program. We wouldn't expect the return value to have any significance. If it does, then you might have a problem. The less you have to remember, and the less other people have to remember while looking at your code, the better. Use naming conventions, even in your own code, to signify which types of functions mutate vs. which functions return values.

The more you have to think about a function, the harder it is to use it. If it has side effects and returns a value, it makes debugging and using the function difficult.

One corollary of this rule is that you should avoid functions that print things and return a useful value. For instance, a `get_and_display_total` function might display a total value and also return it. Those are two distinct actions that you may not always want to do together.

There are exceptions to the "don't print and return" guideline. One exception is a function that requests some input from the user and then returns that input. Logically, that's a single operation: "get input from the user." An example of such a function is the `input` function in Python, which lets you read user input from the terminal.

Your goal should be to build small functions that are like LEGO blocks: they should be stand-alone pieces of functionality that you can use to piece together larger structures. You don't want these functions to be mentally taxing to use. Interesting structures comprise hundreds or thousands of individual LEGO pieces. Likewise, large programs combine hundreds or thousands of small functions. (Later, we'll talk about organizing functions into classes and objects).

Some functions are convoluted since the logic is complex. It's a sign that you don't quite understand the problem well enough to break it down into well-compartmentalized pieces. That's fine at first. However, your understanding should grow as you dig deeper into the code. When it does, refactor your code to reflect that increased clarity.

This process is similar to writing. Your first draft is almost exploratory, dumping out ideas all over the place. As your narrative comes into focus, the structure of your piece becomes more organized and clean.

Displaying Output

Some functions only display information. For example:

Copy Code

```
def welcome():  
    print('welcome')
```

That's fine, but it's not clear whether a function named `welcome` returns a string or outputs a string. One way to resolve this is to help yourself remember and prefix functions that output values with something like `print`, `say` or `display`. That requires some discipline, and it's vital that you only output values in these functions. Don't mutate arguments or return values.

Functions are like black boxes. It takes some stuff (input) and returns some

value (output) to you. They should be self-contained; you should know what they do without looking at the implementation. Coding is much easier if you follow these general guidelines.

Remember that code not only must work correctly but must also be easy to read, both by others as well as your future self.

Miscellaneous Tips

Watch your indentation. Use 4 spaces, not tabs.

Name your functions from the perspective of using them later. That is, think first about how you want to invoke them; think about the implementation later. For example, if you have a list of cards, and you want to write a function to find the ace, your function should be called `find_ace`, and you can use it like this: `ace = find_ace(cards)`. You shouldn't name it `find_ace_from_cards`, since you'd be invoking it like this: `find_ace_from_cards(cards)`. When the reader sees a function like that, they think "What else can you find an ace from other than cards? Tiles?" The easier it is to read your code, the easier it is to debug and maintain.

Know when to use a regular while loop vs a generic while True loop. Here's an example:

Copy Code

```
while answer.lower() != 'n':  
    print('Continue? (y/n)')  
    answer = input()
```

When running this code, Python will throw an exception of "NameError: name 'answer' is not defined". To correct it, we must initialize answer before the while statement, like this:

Copy Code

```
answer = ''  
while answer.lower() != 'n':  
    print('Continue? (y/n)')  
    answer = input()
```

That would certainly work, but another implementation would be to use a generic loop with a break statement:

Copy Code

```
while True:  
    print('Continue? (y/n)')  
    answer = input()  
    if answer.lower() == 'n':  
        break
```

Here, all the code is contained in the loop, and it's slightly easier to reason about it. You could even do without the answer variable and use the user's input (i.e. `input()`) in the if condition directly, but using answer is fine -- remember clarity over terseness.

Approach to Learning

Learning to program takes focus and attention. It takes plenty of repetition over a long period. One of the surprising aspects of learning to program is that it's not a sequential process. You can't master Python, then master Flask, and then master testing. You'll likely revisit topics over and over, and, through experience, the most important information will get burned into your long-term memory.

The first time you encounter a topic, it can be daunting. However, the more often you see it, the easier it becomes to understand and use it. The tips mentioned here may not make sense now, but that's ok. Over time, many of these topics will resurface, and it'll make more sense the second time around. Don't be demoralized if you do it once and you can't remember most of it. That's normal. Keep moving forward, and don't be afraid to spend time gaining valuable

experience.

Variables and Functions

Understanding how variables work together with functions is a key part of reading and writing Python code, but establishing clear mental models for how these parts of the Python language work together can be tricky at first.

Examine and compare the following examples:

Example 1

Copy Code

```
my_var = 1
```

```
def my_func():  
    print(my_var) # Output: 1
```

```
my_func()
```

my_var on line 4 is the same variable as my_var on line 1.

Example 2

Copy Code

```
my_var = 1
```

```
def my_func():  
    my_var = 2
```

```
my_func()
```

```
print(my_var) # Output: 1
```

By default, assigning a variable inside a function, as we do on line 4, creates a new local variable inside that function. Thus, my_var on line 4 is a different variable than the one on lines 1 and 7.

Example 3

Copy Code

```
my_var = [1]
```

```
def my_func():  
    my_var[0] = 2
```

```
my_func()
```

```
print(my_var) # Output: [2]
```

In this code, we're reassigning an element of the list referenced by my_var. Unlike example 2, this does not create a new variable -- reassigning an element in a collection mutates the collection.

Example 4

Copy Code

```
def my_func():  
    my_var = 1
```

```
my_func()
```

```
print(my_var) # NameError: name 'my_var' is not defined
```

As seen in example 2, assigning a variable inside a function creates a new local variable inside the function. Since a local variable is only visible in the function in which it is defined, trying to print my_var on line 5 produces an error.

Example 5

Copy Code

```
my_var = [1]
```

```
def my_func(my_var):  
    my_var.append(2)
```

```
my_func(my_var)
print(my_var) # Output: [1, 2]
```

In this code, `my_var` on lines 3 and 4 is a different variable from `my_var` on lines 1, 6, and 7. However, since we pass `my_var` as an argument to `my_func` on line 6, `my_var` inside the function is a reference to the object that is referenced by `my_var` on line 1. Thus, the mutating method `append` mutates that object, which means that the final output is `[1, 2]`.

Example 6

Copy Code

```
my_var = [1]
```

```
def my_func(my_var):
    my_var = [2]
```

```
my_func(my_var)
print(my_var) # Output: [1]
```

Unlike example 5, the assignment on line 4 makes the function's parameter `my_var` reference a new list `[2]`, while the global `my_var` continues to reference the original list `[1]`.

Example 7

Copy Code

```
my_var = "Hello"
```

```
def my_func():
    print(my_var + " world") # Output: "Hello world"
```

```
my_func()
```

In this code, `my_var` on line 4 is the same variable as `my_var` on line 1. We are not assigning `my_var` inside the function, so we can access the global variable.

Example 8

Copy Code

```
my_var = "Hello"
```

```
def my_func():
    return my_var + " world"
```

```
my_func()
print(my_var) # Output: "Hello"
```

In this example, `my_func` concatenates `my_var` and `" world"` in the same way as example 7. Rather than printing it immediately, it instead returns the result to the calling code on line 6. However, the code on line 6 does not capture the return value. Therefore, `my_var` has its original value when we print it on line 7.

Example 9

Copy Code

```
my_var = "Hello"
```

```
def my_func():
    my_var = my_var + " world"
    # UnboundLocalError: local variable 'my_var' referenced before assignment
    return my_var
```

```
my_func()
print(my_var)
```

In this example, the assignment to `my_var` creates a new local variable inside `my_func`. That new variable is initially undefined, so when the code attempts to append `my_var` and `" world"`, an error occurs.

How would you explain the difference between a function definition and a function invocation?

[Click here to ask LSBot this question](#)

Type your answer to the question above... (Press Cmd/Ctrl+Enter to send)

[Check My Answer](#)

[Summary](#)

The differences between some of the code examples above may seem minor, but the differences in outcome and output when running the code can be quite significant. Understanding exactly why the differences in the code lead to the differences in outcome requires a clear understanding of a number of different concepts and how they work in combination with each other. These concepts are:

Variable Scope

Mutability

Variables as References

Passing Arguments to Functions

At this stage, you've previously encountered each of these concepts earlier in the course material, and may feel that you already have a good understanding of them. Since these concepts are so important, however, we're going to revisit each in turn over the next few assignments before exploring how they work in combination with each other to determine how variables work with functions and blocks.

Variable Scope

One of the trickiest things to understand for a beginner is the concept of scope and how it pertains to accessing variables. A variable's scope is the part of the program that can access that variable by name. This is true in all programming languages. Specifically, variable scoping rules describe how and where the language finds previously defined variables.

In Python, we have two different types of scope: global and local.

Before we discuss variable scope in a bit more detail, refresh your memory by rereading the Scope section from the book. You should also reread the section on global and nonlocal statements in the More Stuff chapter.

Global Scope

In Python, variables initialized outside of any function have a global scope. This means they can be accessed from anywhere in your program, including inside functions.

Copy Code

```
name = 'John'
```

```
def greet():  
    print(f"Hello, {name}!")
```

```
greet() # Output: Hello, John!
```

In the example above, the name variable is initialized in the global scope and is accessible within the greet function.

Local Scope

Function Scope

Functions define a new scope for local variables. You can think of the scope defined by a function as an inner scope. Nested functions define nested scopes. A variable's scope is determined by where it is initialized or declared.

Rule 1: Variables defined in a function are local to that function and cannot be accessed in the outer scope:

Copy Code

```
def my_func():  
    local_var = 10  
    print(local_var)
```

```
my_func() # Output: 10
print(local_var) # Raises NameError: name 'local_var' is not defined
Rule 2: Variables that are defined within a function are local unless explicitly
marked as global or nonlocal:
```

Copy Code

```
def my_func():
    global global_var
    global_var = 20
```

```
my_func()
print(global_var) # Output: 20
```

In this example, the `global` keyword is used inside the `my_func` function to indicate that the `global_var` being modified is the global variable, not a local one. As a result, it is accessible outside of the function.

The `nonlocal` statement's use and behavior is similar to that of `global`. However it only applies in nested functions. We'll discuss `nonlocal` in a later course.

Rule 3: Variables used but not reassigned in a function may be in the outer scope:

Copy Code

```
outer_var = 15
```

```
def my_func():
    print(outer_var)
```

```
my_func() # Output: 15
```

In this example, `outer_var` is used but not reassigned in the `my_func` function, so Python looks for it in the outer scope and successfully finds the value 15.

This behavior lets functions access variables from their containing scope unless those variables are redefined within the function. Keep in mind that this only applies to reading the variable's value; if you attempt to reassign the variable within the function without using the `global` keyword, a new local variable will be created instead.

Rule 4: Peer scopes do not conflict:

Copy Code

```
def func_a():
    a = 'hello'
    print(a)
```

```
def func_b():
    print(a) # NameError: name 'a' is not defined
```

```
func_a()
func_b()
```

Executing `print(a)` on line 6 raises an error since `a` is not in scope in `func_b`. The assignment on line 2 does define a variable named `a`, but that variable's scope is confined to `func_a`. `func_b` cannot access the variable defined in `func_a`. This also means that we could define a separate `a` variable in `func_b` if we wanted. The two `a` variables would have different local scopes and would be independent of each other.

Rule 5: Nested functions have their own scope:

Copy Code

```
a = 1 # first level variable
```

```
def foo(): # second level
    b = 2
```

```
def bar(): # third level
    c = 3
    print(a) # => 1
    print(b) # => 2
    print(c) # => 3

bar()

print(a) # => 1
print(b) # => 2
print(c) # => NameError: name 'c' is not defined
```

foo()

If some of the outputs above surprise you, you should study the code carefully and make sure you understand the rules around inner scope versus outer scope.

Block Scope

Unlike some programming languages, Python does not have a block scope. Variables defined within blocks, such as if statements or loops, are accessible outside of those blocks.

Copy Code

```
if True:
    block_var = 'Hello'
```

```
print(block_var) # Output: Hello
```

In the example above, `block_var` is initialized within the if block but can still be accessed outside of it.

Pass by Reference vs Pass by Value

It's important to know what happens to function arguments. Different languages have different strategies for this. The terms pass-by-reference and pass-by-value describe the two main strategies. This assignment provides an understanding of what those terms mean and how they relate to Python's behavior. In the end, it doesn't matter what you call it, as long as you understand how to invoke functions with the behavior you expect.

What does pass by "value" mean?

In order to talk about what "pass by value" traditionally means, we'll talk about it using a traditional language -- C. In C, when you "pass by value", the function only has a copy of the original object. Operations performed on the object within the function have no effect on the original object outside of the function.

That can lead you to believe that Python is pass-by-value since reassigning a parameter variable within a function doesn't affect the variable outside the function. For example:

Copy Code

```
def change_name(name):
    name = 'bob'
```

```
name = 'jim'
change_name(name)
print(name) # Output: jim
```

Note that the code example above has two different variables called `name`. There is a locally scoped variable within the function, and another in the global scope.

The question is: when `name` from the global scope is passed into the function, via `change_name(name)`, did we pass in a reference or did we pass in the value?

It looks like it was passed by value, since re-assigning the variable only

affected the function-level variable, and not the global scope variable.

What does pass by "reference" mean?

However, it's not quite that simple. If Python was pure "pass by value", that means there should be no way for operations within a function to cause changes to the original object. But you can plainly do this in Python. For example:

Copy Code

```
def add_element(my_list):  
    my_list.append([4])
```

```
my_list = [1, 2, 3]  
add_element(my_list)  
print(my_list)      # => [1, 2, 3, [4]]
```

This implies that Python is "pass by reference", since operations within the function affected the original object. However, as we saw with the re-assignment example, not all operations affect the original object. For example, we'll slightly modify the previous code:

Copy Code

```
def add_element(my_list):  
    my_list = my_list + [4]
```

```
my_list = [1, 2, 3]  
add_element(my_list)  
print(my_list)      # => [1, 2, 3]
```

It appears we're back in "pass by value" world again, where operations within the function don't affect the original. What's going on?

What Python does

As you can see from the previous examples, Python exhibits a combination of behaviors from both "pass by reference" as well as "pass by value". Its behavior is described as pass by object reference as you are actually passing a reference to the object (value) rather than a copy of the object itself. However, whether the function can modify the object depends on whether the object is mutable or immutable.

The important thing to remember is that Python always passes references to objects, not the objects themselves. The distinction between mutable and immutable types in Python then dictates whether you can modify the original object through that reference.

Coding Tips 2

Now that you have a little bit more programming experience under your belt, it's time to talk about a few more coding tips to follow.

Using blank lines to organize code

Blank lines are important visual cues in your program.

Just as writers use paragraphs to organize related sentences, we (that is, programmers) also need to learn to organize chunks of code to make it easier to read.

Making your code readable is of paramount importance, not only for others but for your future self.

Sometimes, that future is just a few hours or days later.

At first, it's going to be challenging to develop a feel for using blank lines in your programs. Over time, you'll get better at it, especially if you read a lot of existing code.

Copy Code

```
# bad
```

```
name = input("Enter your name: ").strip()
while not name:
    print("That's an invalid name. Try again:")
    name = input().strip()
print(f"Welcome {name}!")
print("What would you like to do?")
Use some blank lines to separate the different concerns in the code.
```

Copy Code
better

```
name = input("Enter your name: ").strip()

while not name:
    print("That's an invalid name. Try again:")
    name = input().strip()

print(f"Welcome {name}!")
print("What would you like to do?")
Visually, you can quickly see where the name variable is defined. You can also
see that this small code snippet is roughly divided into 3 parts:
```

```
initial user input
input validation
using the variable
Name functions appropriately
To remind yourself of a function's purpose, choose descriptive function names.
If you have some functions that output values, you might name them with prefixes
like display_ or print_. For instance, seeing a function named print_total, you
can expect it to print a total rather than return anything. This convention
helps to avoid the need to check the function's implementation every time you
use it.
```

If you find yourself looking at a function's implementation every time you use it, it's a sign that the function name needs to be improved.

This advice can be distilled down to: ensure a function does one thing and is named appropriately. A function named total should return a value, while print_total should display something and likely return None.

Don't mutate the collection during iteration
Suppose we have a list of strings and we want to iterate over that list and print out each element. We could do something like this:

```
Copy Code
words = ['scooby', 'do', 'on', 'channel', 'two']

for word in words:
    print(word)
That's a very typical code. Now suppose we want to remove each element as we're
iterating:
```

```
Copy Code
words = ['scooby', 'do', 'on', 'channel', 'two']

for word in words:
    print(word) # prints: scooby, on, two (in that order)
    words.remove(word)
The list.remove method removes the element passed to it as the argument from the
list. Since we're iterating through the list and calling remove in each
iteration, we expect all elements to be removed by the end of the iteration.
However, let's log the list after the iteration to see whether that is indeed
what happens:
```


Copy Code

```
words = ['scooby', 'do', 'on', 'channel', 'two']
```

```
for word in words:
    print(word)
    words.remove(word)
```

```
print(words) # prints: ['do', 'channel']
```

That is very strange -- shouldn't every element be deleted? We're expecting an empty list, but the final value is ['do', 'channel'], which may result in some confusion. To understand what's happening, let's walk through this code one thing at a time.

In the first iteration, word points to the first element of words, namely, 'scooby'. That word gets printed, and it is then removed from words.

At this point, words contains ['do', 'on', 'channel', 'two']

In the second iteration, word now points to the second element of words, which is now 'on'. Thus, at this point, we print 'on'. When we call words.remove(), we remove element 'on' from the list. And this is where the error happens. We have skipped the element 'do' in the iteration since the list that we are iterating on had one element less.

At this point, words contains ['do', 'channel', 'two']

In the third iteration, word now points to the third element of words, which is now 'two'. Thus, at this point, we print two. We call words.remove(), and remove it from the list.

At this point, words contains ['do', 'channel']

Python now attempts to do a fourth iteration. However, since words now only contains two elements, iteration ends.

The lesson here is:

Don't mutate a collection while iterating through it. The behavior may not be what you expect.

Note, however, that you can mutate the individual elements within that collection, just not the collection itself.

Copy Code

```
pairs = [[6, 'scooby'], [2, 'do'], [2, 'on'], [7, 'channel'], [3, 'two']]
```

```
for pair in pairs:
    pair.pop(0)
```

```
print(pairs) # prints [['scooby'], ['do'], ['on'], ['channel'], ['two']]
```

Variable shadowing

Variable shadowing occurs when a local variable in an inner scope shares the same name as a variable in an outer scope. This can easily lead to bugs and confusion since it prevents access to the outer scope variable from within the inner scope.

Let's see an example. Suppose we have a list of names, and we want to append a last name to each of them. We could write some code like this:

Copy Code

```
name = 'johnson'
```

```
def print_full_name():
    for name in ['kim', 'joe', 'sam']:
        print(f"{name} {name}")
```

```
print_full_name()
```

```
# kim kim
```

```
# joe joe
```

```
# sam sam
```

```
print(name) # 'johnson'
```

In this Python example, we've shadowed the outer scoped name variable. Within the for loop, the name variable represents the elements in the list - "kim", "joe", or "sam".

Note that the following is not variable shadowing but a reassignment:

Copy Code

```
name = 'johnson'
```

```
def print_full_name():
    global name
    for name in ['kim', 'joe', 'sam']:
        print(f"{name} {name}")
```

```
print_full_name()
```

```
# Prints:
```

```
# kim kim
```

```
# joe joe
```

```
# sam sam
```

```
print(name) # sam
```

When we print the variable name on the last line, its value is now 'sam'.

The best approach would be to rename the for loop variable to something more precise like first_name:

Copy Code

```
name = 'johnson'
```

```
def print_full_name():
    for first_name in ['kim', 'joe', 'sam']:
        print(f"{first_name} {name}")
```

```
print_full_name()
```

```
# Prints:
```

```
# kim johnson
```

```
# joe johnson
```

```
# sam johnson
```

```
print(name) # johnson
```

It's crucial to be cautious about selecting appropriate variable names, especially when using loops or nested functions. If you choose a variable name that matches an outer scope variable, you risk shadowing the outer variable.

Use underscore for unused identifiers

Suppose you have a list of names, and you want to print out a string for every name in the list, but you don't care about the actual names. In those situations, use an underscore to signify that we don't care about this particular loop variable.

Copy Code

```
names = ['kim', 'joe', 'sam']
```

```
for _ in names:
```

```
    print('Got a name!')
```

Another example is when you need the first item (typically the index) of a sequence in a loop but don't need the second item (the value). You can use _ to indicate that the second item is not being used in the loop:

Copy Code

```
names = ['kim', 'joe', 'sam']
```

```
for index, _ in enumerate(names):
```

```
    print(f"{index}: Got a name!")
```

```
# prints
```

```
# 0: Got a name!
```

```
# 1: Got a name!
```

2: Got a name!

Note: The enumerate function is used to iterate over a sequence in a way that makes both the index and the value available to the loop body. Thus, the above loop iterates over names, with each iteration passing the index to the index variable, and the element value to the _ variable.

Gain experience through struggling

The final tip in this assignment is about dealing with struggling. There are two things that beginners feel at this stage:

that they must know the "best" or "right" way to do something, so they feel they must learn "best practice."

that they waste too much time debugging and not doing things correctly the first time

To the first point: it's less impactful to learn "best practices" without first learning why they are best practices. That leads to the second point, which is that you must learn to be ok with struggling through the "bad" or sub-optimal practices first. That's not wasting time, that's getting experience. Becoming a good developer means experiencing and solving a lot of weird issues.

In other words, don't memorize "best practices," but spend enough time programming to the point where you understand the context for those practices. If you do that, you'll be able to lean on your experience rather than try to retrieve a bullet list from something you read months or years ago.

Therefore, don't fear violating rules or be afraid to make mistakes. However, keep an eye out for improvements. Coding is like writing -- there are syntactical rules, but there are also creative ways of expression.

We can't say this enough: spend time programming. Learn to debug problems, struggle with them, search for the right terms, play around with the code, and you'll be able to transform into a professional developer. That's what professional developers do every day.